



CS4001NI Programming

60% Individual Coursework

2025 Spring

Student Name: Sadikshya Lamichhane

London Met ID: 24046857

College ID: NP01CP4A240293

Assignment Due Date: Friday, May 16, 2025

Assignment Submission Date: Friday, May 16, 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded

# 24046857 Sadikshya Lamichhane 2.docx

Islington College,Nepal

## Document Details

Submission ID

trn:oid:::3618:96171491

Submission Date

May 16, 2025, 10:18 AM GMT+5:45

Download Date

May 16, 2025, 10:20 AM GMT+5:45

File Name

24046857 Sadikshya Lamichhane 2.docx

File Size

27.4 KB

54 Pages

3,657 Words

20,987 Characters



Page 2 of 58 - Integrity Overview

Submission ID trn:oid:::3618:96171491

## 14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

### Match Groups

- 33 Not Cited or Quoted 14%**  
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**  
Matches that are still very similar to source material
- 0 Missing Citation 0%**  
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**  
Matches with in-text citation present, but no quotation marks

### Top Sources

- 6% Internet sources**
- 1% Publications**
- 13% Submitted works (Student Papers)**

### Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

## Match Groups

- 33 Not Cited or Quoted 14%**  
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**  
Matches that are still very similar to source material
- 0 Missing Citation 0%**  
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**  
Matches with in-text citation present, but no quotation marks

## Top Sources

- 6% Internet sources
- 1% Publications
- 13% Submitted works (Student Papers)

## Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

|          |                                     |     |
|----------|-------------------------------------|-----|
| <b>1</b> | Submitted works                     |     |
|          | iacademy on 2025-05-14              | 5%  |
| <b>2</b> | Internet                            |     |
|          | www.coursehero.com                  | 2%  |
| <b>3</b> | Submitted works                     |     |
|          | islingtoncollege on 2025-05-16      | 1%  |
| <b>4</b> | Submitted works                     |     |
|          | Valencia College on 2024-05-21      | <1% |
| <b>5</b> | Submitted works                     |     |
|          | Milton Keynes College on 2025-01-29 | <1% |

## List of Images

|                                      |                                     |
|--------------------------------------|-------------------------------------|
| Image 1 java .....                   | 9                                   |
| Image 2 desktop application .....    | 10                                  |
| Image 3 web application .....        | 11                                  |
| Image 4 mobile application .....     | 11                                  |
| Image 5 enterprise application ..... | 12                                  |
| Image 6 blue j .....                 | 13                                  |
| Image 7 ms word .....                | 14                                  |
| Image 8 balsamiq .....               | 14                                  |
| Image 9 wireframe .....              | 21                                  |
| Image 10 home page .....             | <i>Error! Bookmark not defined.</i> |
| Image 11 main system .....           | 22                                  |
| Image 12 class diagram .....         | 26                                  |
| Image 13 test 1 .....                | 49                                  |
| Image 14 Test 2 .....                | 51                                  |

|                                          |    |
|------------------------------------------|----|
| Image 15 Test 2 .....                    | 54 |
| Image 16 Test 3 .....                    | 57 |
| Image 17 Test 4 .....                    | 62 |
| Image 18 Test 4 .....                    | 65 |
| Image 19 Test 4 .....                    | 69 |
| Image 20 Test 5 .....                    | 73 |
| Image 21 Syntax Error .....              | 77 |
| Image 22 Syntax error correction .....   | 77 |
| Image 23 Run time error .....            | 79 |
| Image 24 run time error correction ..... | 79 |
| Image 25 logical error .....             | 80 |
| Image 26 logical error correction .....  | 80 |

# Table of Content

|                                  |    |
|----------------------------------|----|
| Image 1 java .....               | 9  |
| 1. Desktop Applications .....    | 10 |
| 2. Web Application: .....        | 11 |
| 3. Mobile Application: .....     | 11 |
| 4. Enterprise Applications ..... | 12 |
| Image 6 blue j .....             | 13 |
| Image 7 ms word .....            | 14 |
| Image 8 balsamiq .....           | 14 |
| 1. Class .....                   | 15 |
| 2. Object: .....                 | 15 |
| 3. Method .....                  | 16 |
| 4. Constructor .....             | 16 |
| 5. Inheritance .....             | 16 |
| 6. Polymorphism .....            | 17 |
| 7. Encapsulation: .....          | 17 |
| 8. Abstraction .....             | 18 |
| 9. Interface .....               | 18 |
| 10. Package .....                | 19 |
| 11. Array List .....             | 19 |
| 12. Swing .....                  | 19 |
| 14. This Keyword .....           | 20 |

|                                                   |    |
|---------------------------------------------------|----|
| 15. Super Keyword .....                           | 20 |
| Image 9 wireframe .....                           | 21 |
| Image 11 main system .....                        | 22 |
| Image 12class diagram .....                       | 26 |
| Gym Member Class .....                            | 28 |
| Regular Member Class .....                        | 31 |
| Premium Member Class .....                        | 34 |
| GymGUI Class .....                                | 37 |
| Adding Premium Member .....                       | 53 |
| Mark Attendance .....                             | 56 |
| Test 4 .....                                      | 61 |
| Calculate discount, pay due, revert members ..... | 61 |
| Error Detection and Correction .....              | 77 |
| Conclusion .....                                  | 81 |

## Aim and Objectives

This computing coursework is specially designed to introduce students to the fundamental principles of programming using Java. Typically, Java coursework aims to develop your programming skills while demonstrating your understanding of core Java concepts.

### Core Aims

- Develop proficiency in object-oriented programming using Java
- Apply theoretical concepts to practical programming problems
- Demonstrate code organization and documentation skills
- Build software that meets specific requirements

### Learning Objectives

- Master Java syntax and language fundamentals
- Implements object-oriented Principles (encapsulation, inheritance, polymorphism)
- Design and create classes with appropriate relationships
- Develop error handling and exception management
- Implement data structures and algorithms
- Create user interface
- Apply proper testing Methods.



## Introduction



Image 1 java

Java is a high-level, object-oriented programming language developed by sun Microsystems in 1995. It is widely used due to its platform independence (via the java Virtual Machine), robustness, and security features. Java supports multi-threading, automatic memory management (garbage collection), and a rich standard library that makes it ideal for building a variety of applications, from console programs to complex GUI-based systems.

Java's Swing framework provides tools for building graphical user interface, allowing developers to create interactive desktop applications. This coursework utilizes Java's object-oriented principles and GUI capabilities to develop a comprehensive system for managing gym membership.

## Application Of Java

Java is one of the most widely used programming languages in the world. Due to its platform independence, object-oriented nature, and rich libraries, java is used in many fields across industries.

### 1. Desktop Applications

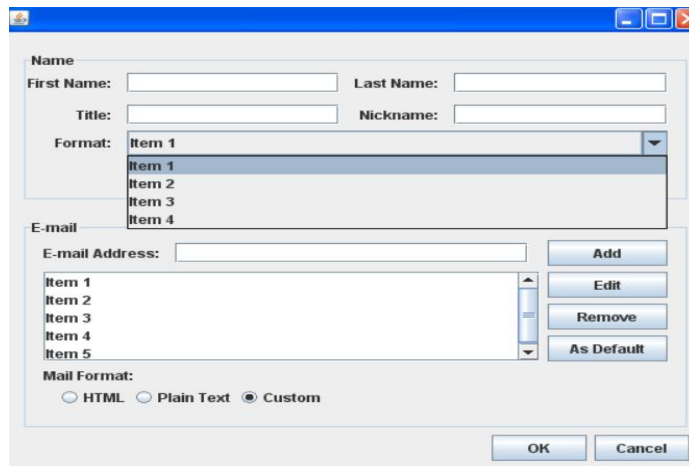


Image 2 desktop application

Java, especially with swing and JavaFX, is commonly used to build graphical user interface (GUI) applications such as:

- Gym Management System
- Billing Inventory Software
- Media Players
- Scientific Calculators

For Example: Gym Membership System

## 2. Web Application:



Image 3 web application

Java is used for building server-side applications using technologies like, Spring Framework and JSP (Java Server Pages).

For Example:

- E-Commerce
- Website
- Content Management System (CMS), etc.

## 3. Mobile Application:



Image 4 mobile application

Java was the official language for android development for many years. It is still widely used to create Android apps and mobile games.

For Example:

- WhatsApp (initial versions)
- Many android utility apps.

#### 4. Enterprise Applications



Image 5 enterprise application

Java is highly favored in enterprise-level applications (banking, HR, billing, etc.) because of its robustness, scalability, and security.

Example:

- Banking system
- payroll system
- ERP system.

## Blue J



Image 6 blue j

BlueJ is a free, simple and beginner-friendly Java Integrated Development Environment (IDE) designed especially for students and new Java programmers. It was developed by Michael Kolling and John Rosenberg at the University of Kent.

### Features

- It is user friendly
- It has object visualization
- It runs Interactive Execution
- It is light weight and Portable (Runs on multiple platforms).

It is ideal for learning Object-Oriented Programming (OOP) concepts like classes, objects, inheritance, and polymorphism. It encourages visual learning by showing how java components are connected. It's also suitable for writing small to medium programs like: Student record systems, Gym Member Management (Like coursework), and simple games or tools.

## Ms Word

Image 7 ms word



Microsoft Word is a word processing program that allows for the creation of both simple and complex documents. With Office 365, we can download the application to the hard drive and will also have access to the online version.

Ms Word enables users to do write-ups, create documents, resumes, contacts, etc. It helps with the creation, editing, and formatting of documents. It includes facilities for spelling and grammar checking, as well as text formatting. We can also include photos, tables and charts in your documents.

## Balsamiq

Image 8 balsamiq



**balsamiq Wireframes**

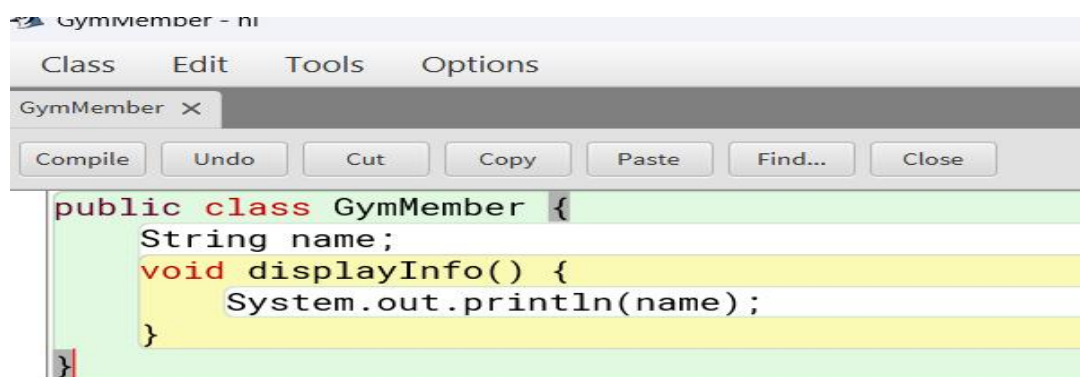
Balsamiq wireframe is a user interface design tool for creating wireframes (sometimes called mockups or low-fidelity prototypes). You can use it to generate digital sketches of your idea or concept for an application or website, to facilitate discussion and understanding before any code is written.

## Terms and Terminology

Here are the terms and terminologies used in Java:

### 1. Class

A class is a blueprint for creating objects. It defines variables (fields) and methods.



### 2. Object:

An object is an instance of class that holds actual data and can cause the methods defined in its class.



### 3. Method

Method is a block of code that performs a specific task. It can return value or be void.

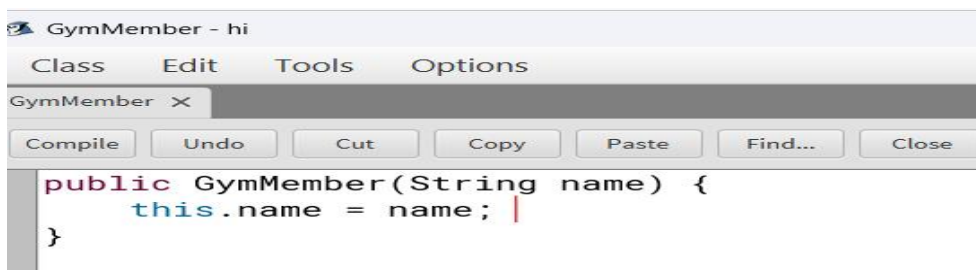


The screenshot shows an IDE window titled 'activatemembership - hi'. The menu bar includes 'Class', 'Edit', 'Tools', and 'Options'. Below the menu bar is a toolbar with buttons for 'Compile', 'Undo', 'Cut', 'Copy', 'Paste', 'Find...', and 'Close'. The main editor area displays the following Java code:

```
public void activatemembership() {  
    status = "Active";  
}
```

### 4. Constructor

Constructor is a special method used to initialize new object. It has the same name as the class and no return type.

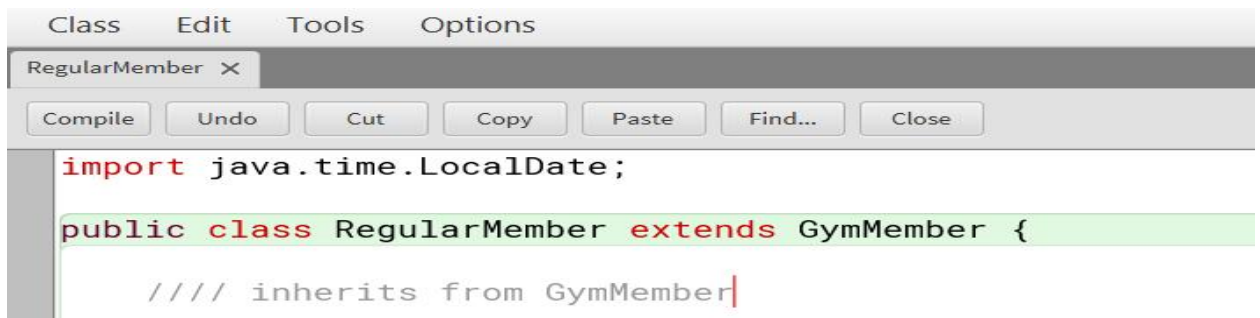


The screenshot shows an IDE window titled 'GymMember - hi'. The menu bar includes 'Class', 'Edit', 'Tools', and 'Options'. Below the menu bar is a toolbar with buttons for 'Compile', 'Undo', 'Cut', 'Copy', 'Paste', 'Find...', and 'Close'. The main editor area displays the following Java code:

```
public GymMember(String name) {  
    this.name = name;  
}
```

### 5. Inheritance

When a class inherits properties and method from another class using extends then it is known as inheritance.



The screenshot shows an IDE window titled 'RegularMember - hi'. The menu bar includes 'Class', 'Edit', 'Tools', and 'Options'. Below the menu bar is a toolbar with buttons for 'Compile', 'Undo', 'Cut', 'Copy', 'Paste', 'Find...', and 'Close'. The main editor area displays the following Java code:

```
import java.time.LocalDate;  
  
public class RegularMember extends GymMember {  
    //// inherits from GymMember
```



## 6. Polymorphism

Polymorphism is the ability to process differently depending on their class type (e.g., overriding methods).

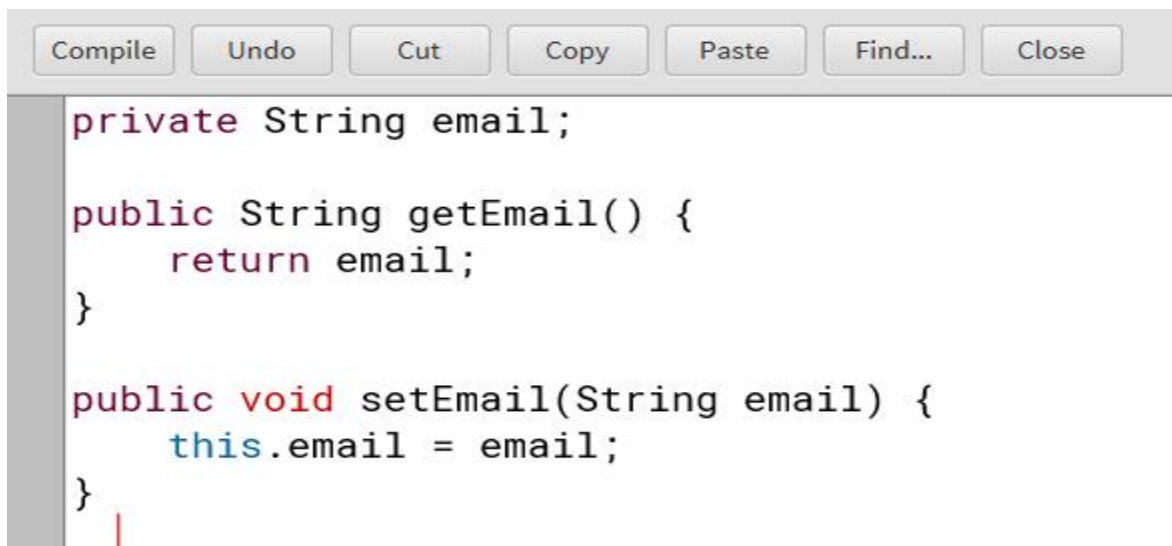


The screenshot shows an IDE window titled "RegularMember - hi". The menu bar includes "Class", "Edit", "Tools", and "Options". Below the menu bar is a tab labeled "RegularMember X". A toolbar contains buttons for "Compile", "Undo", "Cut", "Copy", "Paste", "Find...", and "Close". The code editor displays the following Java code:

```
public void displayInfo() {  
    System.out.println("Regular Member");  
}
```

## 7. Encapsulation:

Encapsulation is the concept of wrapping variables and method together in one unit (class) and using private access with get/ set methods.

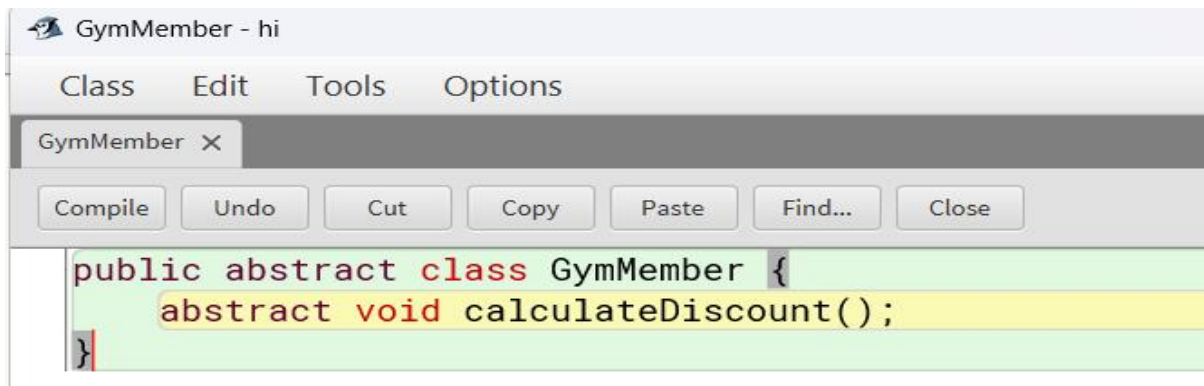


The screenshot shows an IDE window with a toolbar containing buttons for "Compile", "Undo", "Cut", "Copy", "Paste", "Find...", and "Close". The code editor displays the following Java code illustrating encapsulation:

```
private String email;  
  
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}
```

## 8. Abstraction

Abstraction is used for hiding internal implementation details and showing only the necessary features.

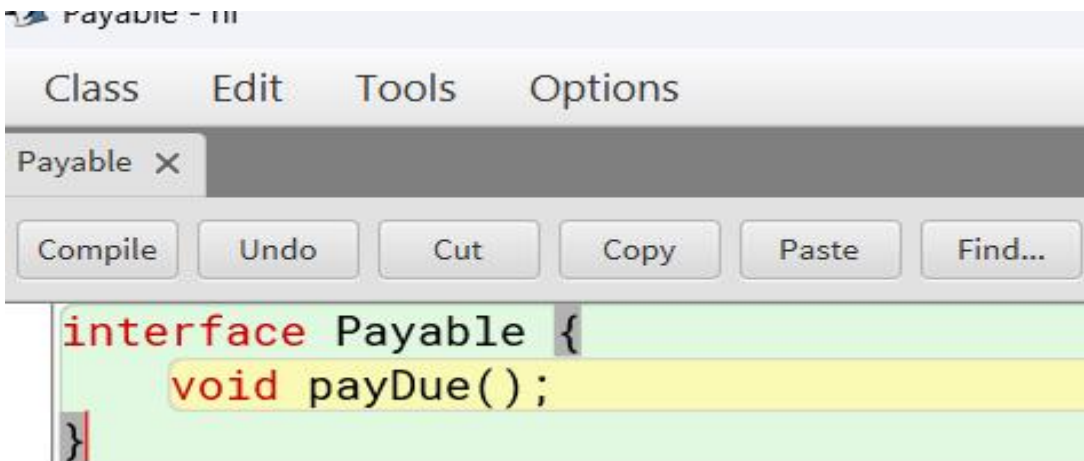


The screenshot shows an IDE window titled "GymMember - hi". The menu bar includes "Class", "Edit", "Tools", and "Options". Below the menu bar is a toolbar with buttons for "Compile", "Undo", "Cut", "Copy", "Paste", "Find...", and "Close". The main editor area displays the following Java code:

```
public abstract class GymMember {  
    abstract void calculateDiscount();  
}
```

## 9. Interface

Interface is a reference type similar to a class that can contain only constants and abstract methods (until Java 8).

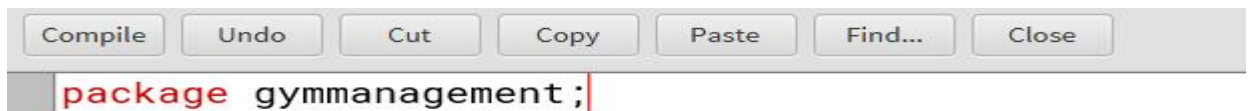


The screenshot shows an IDE window titled "Payable - hi". The menu bar includes "Class", "Edit", "Tools", and "Options". Below the menu bar is a toolbar with buttons for "Compile", "Undo", "Cut", "Copy", "Paste", and "Find...". The main editor area displays the following Java code:

```
interface Payable {  
    void payDue();  
}
```

## 10. Package

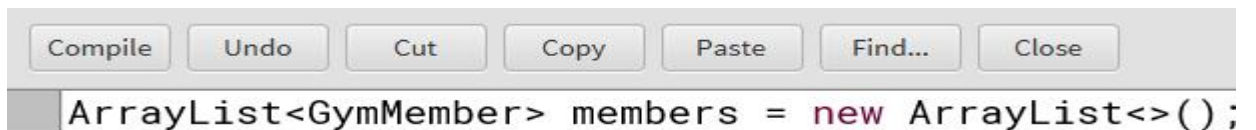
Package is a name space that organizes classes and interfaces.



A screenshot of an IDE window with a toolbar containing buttons for Compile, Undo, Cut, Copy, Paste, Find..., and Close. Below the toolbar, the text `package gymmanagement;` is displayed in a monospaced font, with the cursor positioned at the end of the line.

## 11. Array List

Array List is a resizable array used to store lists of objects.



A screenshot of an IDE window with a toolbar containing buttons for Compile, Undo, Cut, Copy, Paste, Find..., and Close. Below the toolbar, the text `ArrayList<GymMember> members = new ArrayList<>();` is displayed in a monospaced font, with the cursor positioned at the end of the line.

## 12. Swing

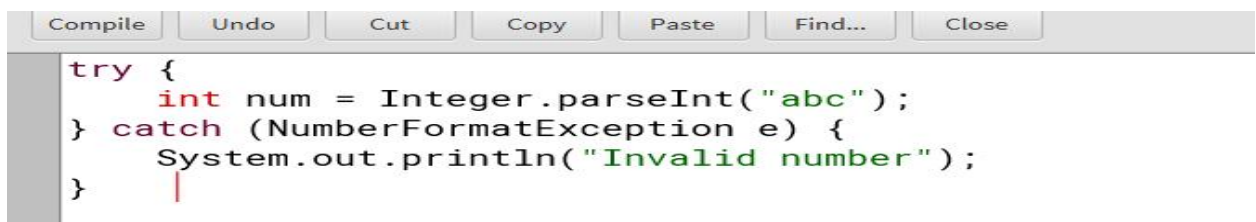
Swing is a GUI toolkit creating desktop applications.



A screenshot of an IDE window with a toolbar containing buttons for Compile, Undo, Cut, Copy, Paste, Find..., and Close. Below the toolbar, the text `JFrame frame = new JFrame("Gym GUI");` is displayed in a monospaced font, with the cursor positioned at the end of the line.

## 13. Try-catch (Exception Handling)

Try-Catch is used to handle errors without crashing the program.



A screenshot of an IDE window with a toolbar containing buttons for Compile, Undo, Cut, Copy, Paste, Find..., and Close. Below the toolbar, a try-catch block is displayed in a monospaced font. The code is as follows:  
`try {  
 int num = Integer.parseInt("abc");  
} catch (NumberFormatException e) {  
 System.out.println("Invalid number");  
}`  
The cursor is positioned at the end of the last line.

#### 14. This Keyword

This Keyword refers to the current object.

```
this.name = name;
```

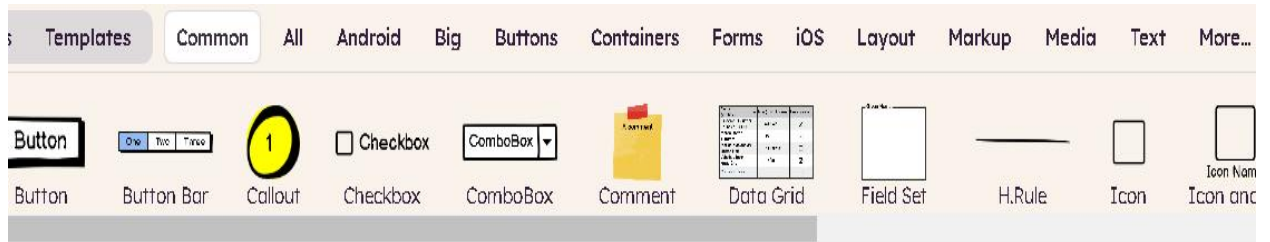
#### 15. Super Keyword

Super Keyword refers to the parent class constructor or method.

```
super.displayInfo();
```

## WIREFRAME

### Sample GUI



### SAMPLE GUI

|                                               |                                                 |                                               |                                   |
|-----------------------------------------------|-------------------------------------------------|-----------------------------------------------|-----------------------------------|
| ID :                                          | <input type="text"/>                            | TOTAL SALES :                                 | <input type="text"/>              |
| NAME :                                        | <input type="text"/>                            | TOTAL DISCOUNTS :                             | <input type="text"/>              |
| LOCATION :                                    | <input type="text"/>                            | MARKED PRICE :                                | <input type="text"/>              |
| OPENING HOUR :                                | <input type="text"/>                            | SELLING PRICE :                               | <input type="text"/>              |
| PRODUCT NAME :                                | <input type="text"/>                            | VAT INCLUSIVE PRICE :                         | <input type="text"/>              |
| LOYALTY POINTS :                              | <input type="text"/>                            | PURCHASED YEAR :                              | <input type="text" value="2025"/> |
| <br>                                          |                                                 |                                               |                                   |
| <input type="checkbox"/> IS PAYMENT ONLINE    | <input type="button" value="ADD DEPARTMENT"/>   | <input type="button" value="ADD RETAILER"/>   |                                   |
| <input type="checkbox"/> IS IN SALES          | <input type="button" value="DISCOUNTED PRICE"/> | <input type="button" value="LOYALTY POINTS"/> |                                   |
| <input type="button" value="REMOVE PRODUCT"/> | <input type="button" value="DISPLAY"/>          | <input type="button" value="CLEAR"/>          |                                   |

Image 9 wireframe

## Developed Program

### Main System

**Member Information**

Member ID:

Name:

Location:

Phone:

Email:

Gender: ☐ Male ☐ Female

Date of Birth:

Membership Start Date:

Referral Source:

Plan:

Regular Plan Price:

Premium Plan Price:

Paid Amount:

Discount Amount:

Removal Reason:

Trainer's Name:

**Actions**

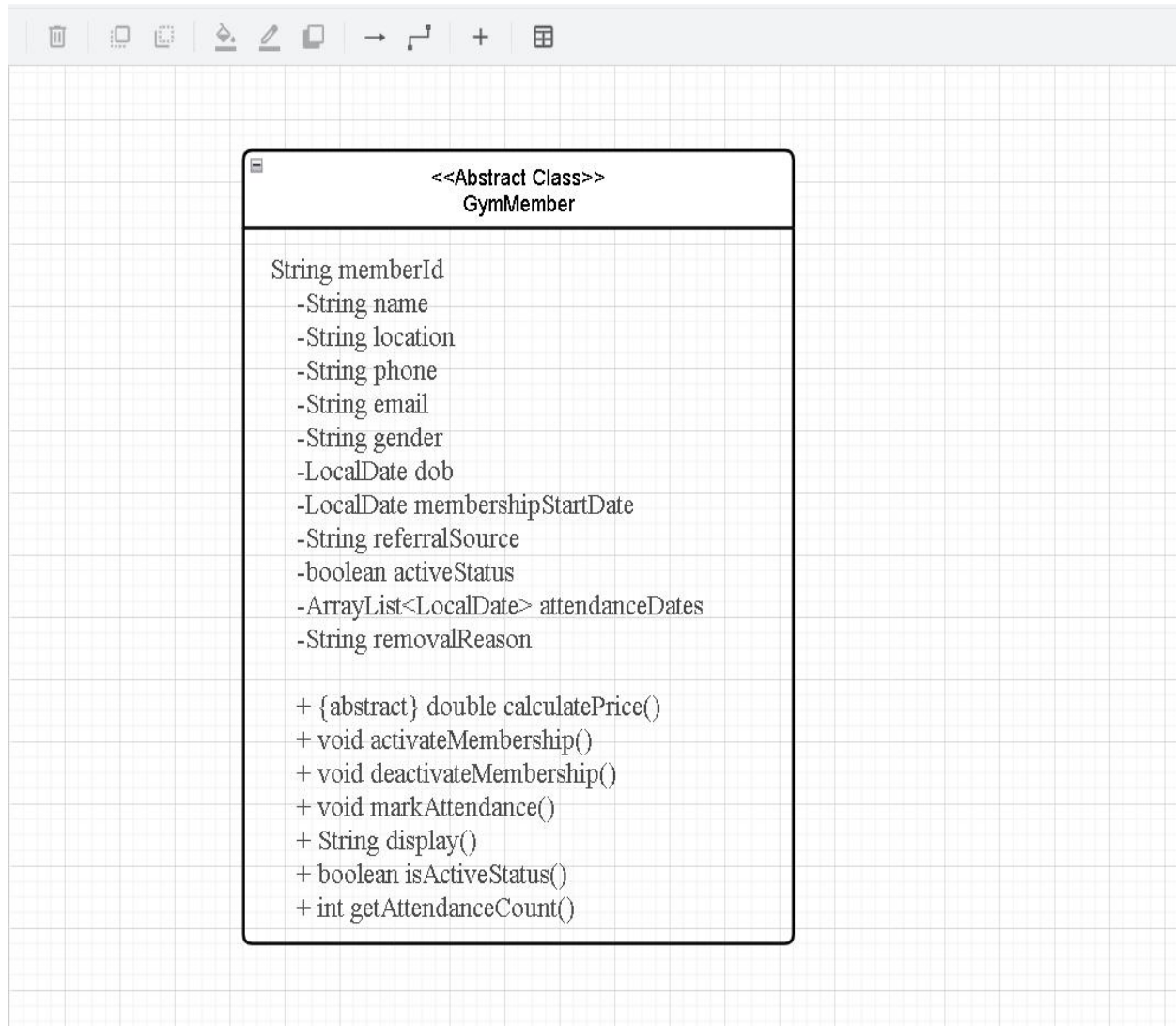
|                       |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |
| Save to File          | Read from File        |                       |

Image 11 main system

## Class Diagram

### Abstract Class

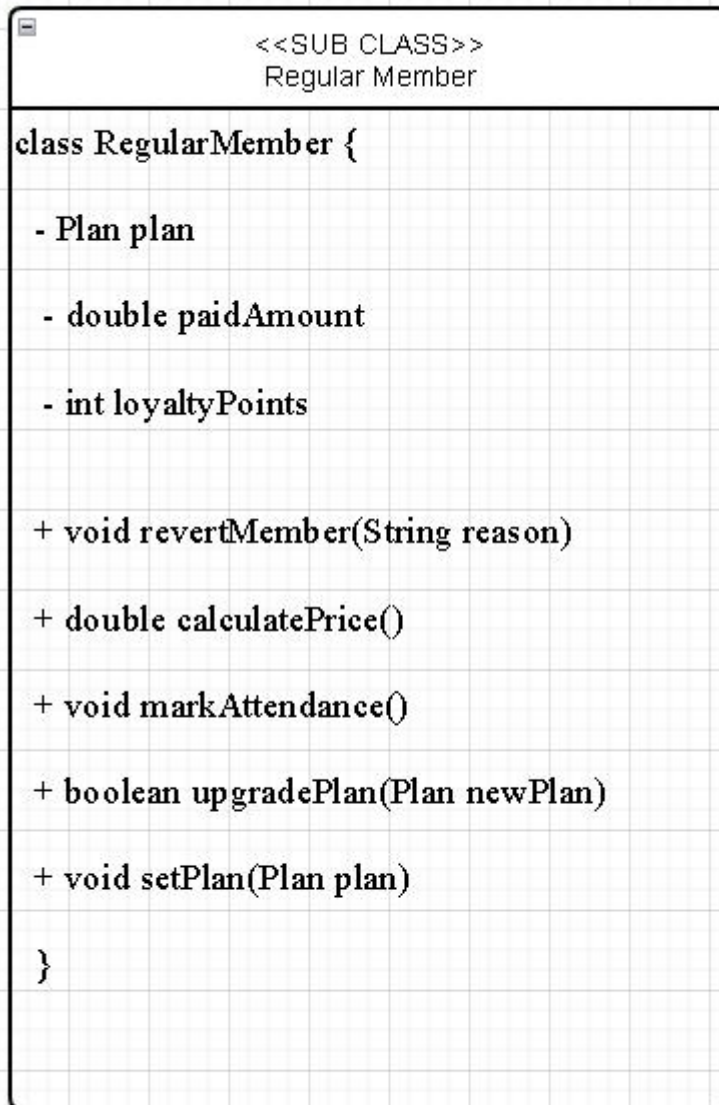
### GymMember





## SubClass

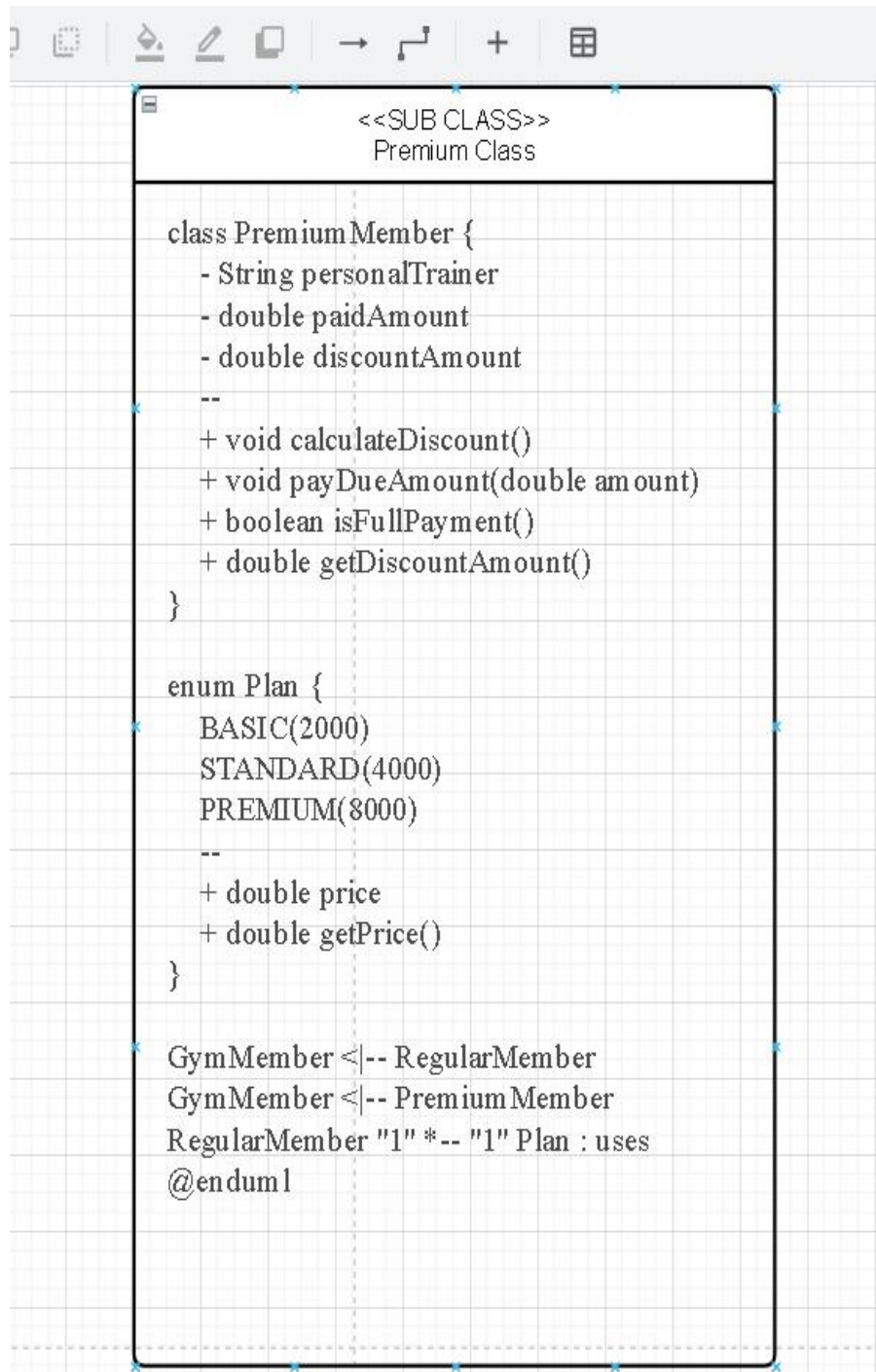
## Regular Class





## SUBCLASS

## PREMIUM CLASS



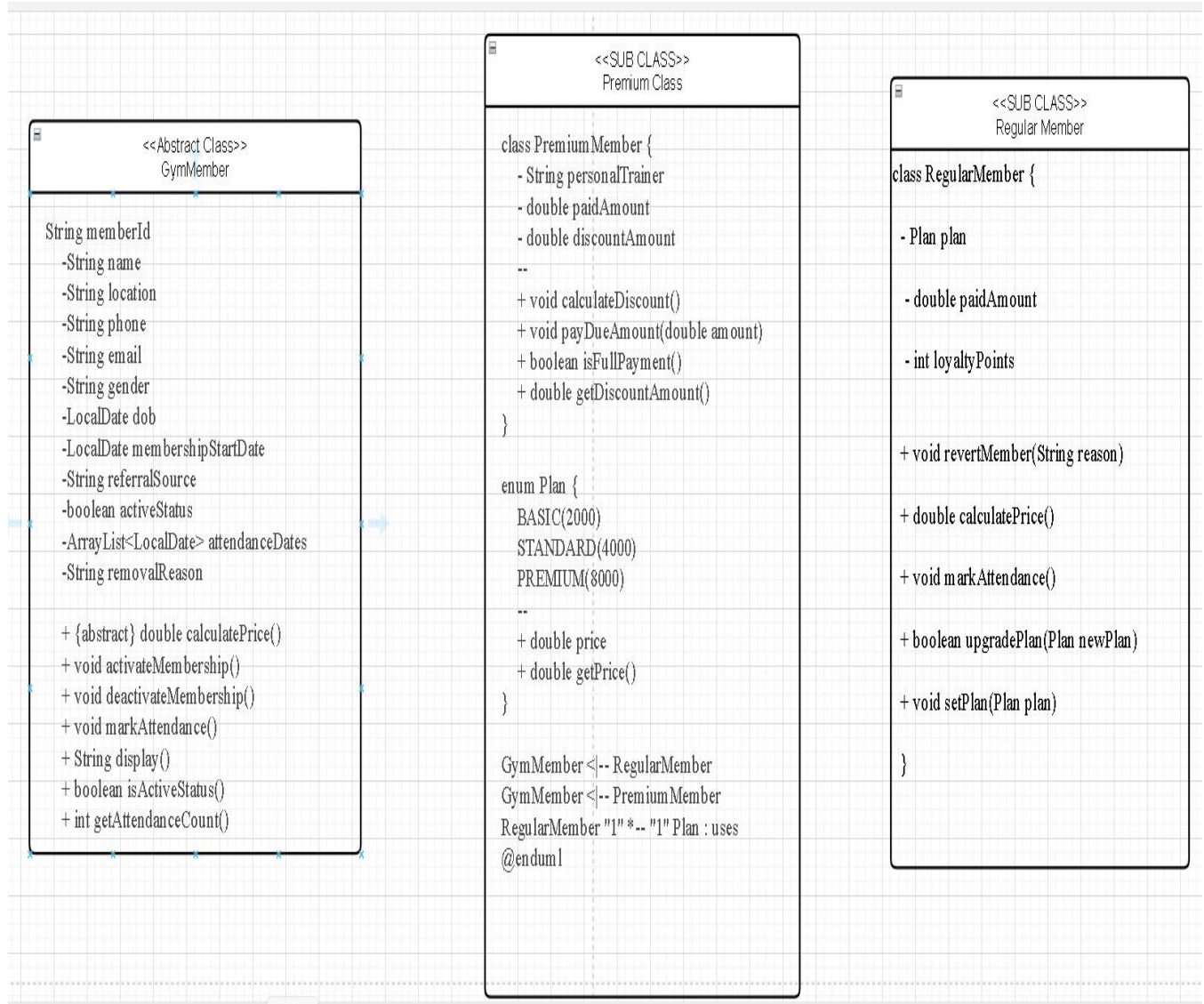
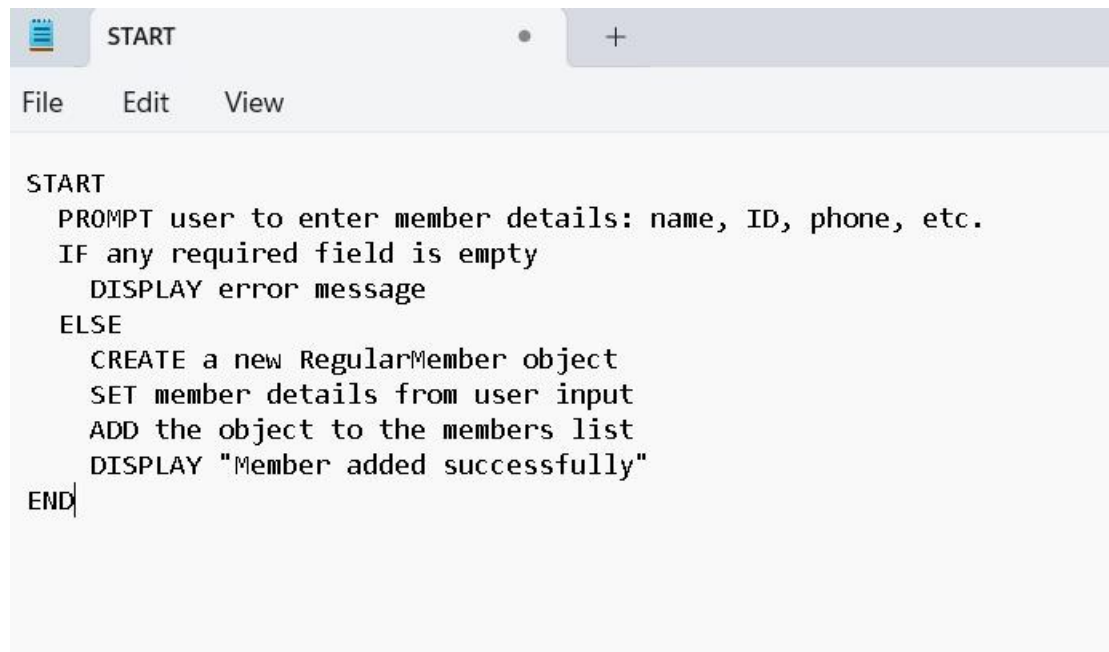


Image 12class diagram

## Pseudocode

Pseudocode is a high level, human readable description of a program's logic or algorithm written in plain language. It's not actual code, but it helps plan the structure of a java program before writing real code.

Pseudocode is language independent, but often follows the style of the programming language you're working in. It plans the logic before actual coding. It helps in reducing errors by thinking through the process. It makes debugging easier by breaking problems into steps.



```
START
  PROMPT user to enter member details: name, ID, phone, etc.
  IF any required field is empty
    DISPLAY error message
  ELSE
    CREATE a new RegularMember object
    SET member details from user input
    ADD the object to the members list
    DISPLAY "Member added successfully"
  END
```

Implementation of every code in detail

× **Gym Member Class**

- Class Definition;

START FUNCTION

DEFINE ABSTRACT CLASS GymMember

DECLARE attributes:

memberId, name, location, phone, email, gender

dob, membershipStartDate, referralSource

activeStatus ← true

attendanceDates ← empty list

RemovalReason

END FUNCTION

- Constructor

START FUNCTION

GymMember(...)

SET memberId, name, location, phone, email, gender, dob,  
membershipStartDate, referralSource

SET activeStatus to true

INITIALIZE attendanceDates as new empty list

END FUNCTION

- Abstract Method

DECLARE ABSTRACT FUNCTION calculatePrice()

- Display Method

START FUNCTION

Display ()

RETURN formatted string with all member information

END FUNCTION

- Activate Membership

START FUNCTION

ActivateMembership ()

IF activeStatus is false THEN

SET activeStatus to true

PRINT "Membership activated for member"

ELSE

PRINT "Membership is already active"

END FUNCTION

- Deactivate Membership

START FUNCTION

deactivateMembership()

IF activeStatus is true THEN

SET activeStatus to false

PRINT "Membership deactivated for member"

ELSE

PRINT "Membership is already inactive"

END FUNCTION

- Mark Attendance

START FUNCTION markAttendance()

IF activeStatus is true THEN

GET today's date

ADD today's date to attendanceDates list

PRINT "Attendance marked"

ELSE

PRINT "Cannot mark attendance for inactive member"

END FUNCTION

- Set and Get Removal Reason

```
START FUNCTION setRemovalReason(reason)
    SET removalReason to reason
END FUNCTION
```

```
START FUNCTION getRemovalReason()
    RETURN removalReason
END FUNCTION
```

- Other Getters

```
START FUNCTION
    getMemberId(), getName(), getLocation(), getPhone(), getEmail(),
        getGender(), getDob(), getMembershipStartDate(), getReferralSource(),
        getAttendanceDates(), getAttendanceCount()
    RETURN corresponding value
END FUNCTION
```

- To String Method

```
START FUNCTION
    toString()
```

```
RETURN memberId, name, and membership type (hardcoded as "Regular")
```

```
END FUNCTION
```

## × Regular Member Class

- Class Definition

```
START FUNCTION
DEFINE CLASS RegularMember EXTENDS GymMember
    DEFINE ENUM Plan with BASIC(2000), STANDARD(4000),
    PREMIUM(8000)
    DECLARE VARIABLES:
        plan, paidAmount, loyaltyPoints ← 0
    SET DISCOUNT_RATE ← 5%
END FUNCTION
```

- Constructor

```
START FUNCTION
RegularMember(...)
    CALL superclass constructor with basic member details
    SET plan
    SET paidAmount
    SET loyaltyPoints to 0
END FUNCTION
```

- RevertMember(reason)

```
START FUNCTION
RevertMember (reason)
    SET plan to BASIC
    SET paidAmount to 0
    SET loyaltyPoints to 0
    CLEAR attendanceDates
    SET activeStatus to false
    SET removalReason to reason
    PRINT "Member reverted with reason"
END FUNCTION
```

- CalculatePrice ()

```
START FUNCTION
```

```

CalculatePrice ()
    SET basePrice ← price of current plan
    IF attendance count > 10 THEN
        APPLY 5% discount to basePrice

    RETURN basePrice
END FUNCTION

```

- Mark Attendance

```

START FUNCTION
MarkAttendance ()
    IF member is active THEN
        GET today's date

        IF today's date not in attendanceDates THEN
            ADD today to attendanceDates
            INCREMENT loyaltyPoints by 1

            IF attendance count equals 11 THEN
                ADD 5 bonus loyaltyPoints

            PRINT "Attendance marked with loyalty points"
        ELSE
            PRINT "Attendance already marked today"
        ELSE
            PRINT "Cannot mark attendance for inactive member"
    END FUNCTION

```

- UpgradePlan(new Plan)

```

START FUNCTION
UpgradePlan (newPlan)
    IF newPlan is of higher rank than current plan THEN
        SET plan to newPlan
        SET paidAmount to newPlan's price
        PRINT "Plan upgraded"
        RETURN true
    ELSE

```



```
        PRINT "Selected plan is not higher"
        RETURN false
    END FUNCTION
```

- Display () Method

```
START FUNCTION
Display ()
    CALL super.display()
    DISPLAY membership type as Regular
    DISPLAY plan name
    DISPLAY paidAmount and calculated price
    DISPLAY attendance count and loyalty points
    DISPLAY active status
END FUNCTION
```

- Getters and Setters

```
FUNCTION getPlan(), setPlan(plan)
FUNCTION getPaidAmount(), setPaidAmount(amount)
FUNCTION getLoyaltyPoints(), setLoyaltyPoints(points)
```

## × Premium Member Class

- Class Definition

```
DEFINE CLASS PremiumMember EXTENDS GymMember
    DECLARE CONSTANT PREMIUM_PRICE = 5000
    DECLARE personalTrainer, paidAmount = 0, discountAmount = 0
```

- Constructor

```
START FUNCTION
PremiumMember (...)
    CALL superclass constructor with basic details
    SET personalTrainer
    SET paidAmount = 0
    SET discountAmount = 0
END FUNCTION
```

- CalculateDiscount()

```
START FUNCTION
CalculateDiscount ()
    IF member is active THEN
        GET attendanceCount

        IF attendanceCount > 20 THEN
            SET discountAmount = 15% of PREMIUM_PRICE
        ELSE IF attendanceCount > 10 THEN
            SET discountAmount = 5% of PREMIUM_PRICE
        ELSE
            SET discountAmount = 0

        PRINT discount calculated for member
```

```

ELSE
    PRINT cannot calculate discount for inactive member
END FUNCTION

```

- PayDueAmount (amount)

```

START FUNCTION
PayDueAmount (amount)
    IF member is active THEN
        IF amount > 0 THEN
            ADD amount to paidAmount
            PRINT payment received
        ELSE
            PRINT invalid payment
        ELSE
            PRINT cannot receive payment for inactive member
    END FUNCTION

```

- RevertMember(reason)

```

START FUNCTION
RevertMember (reason)
    CALL deactivateMembership()
    PRINT member reverted with reason
END FUNCTION

```

- Calculate Price ()

```

START FUNCTION
CalculatePrice ()
    RETURN PREMIUM_PRICE - discountAmount
END FUNCTION

```

- IsFullPayment ()

```

START FUNCTION
IsFullPayment ()
RETURN paidAmount ≥ calculatePrice()

```

END FUNCTION

- GetRemainingAmount ()

START FUNCTION

GetRemainingAmount ()

RETURN calculatePrice() - paidAmount

END FUNCTION

- Display ()

START FUNCTION

Display ()

DISPLAY member details:

- ID, Name, Email, Phone
- Plan = Premium
- Start Date, Gender, DOB, Referral Source
- Trainer name
- Discount and Paid Amount

END FUNCTION

## × **GymGUI Class**

- Class Definition

### START FUNCTION

Create JFrame and JPanel for layout

Set title, size, and close operation

Add input fields for:

ID, Name, Location, Phone, Email, Referral Source

Gender (Male/Female)

Date of Birth, Membership Start Date

Paid Amount, Removal Reason

Trainer Name (Premium only)

Add combo box for Plan (Basic, Standard, Premium)

Add price and discount labels (non-editable)

Add buttons for all operations

Add action listeners to each button

Initialize empty member list (ArrayList of GymMember)

Display the GUI

END FUNCTION

- Add Regular Member

### START FUNCTION

METHOD addRegularMember:

Get input values from form fields

Validate that no field is empty and phone number is 10 digits

Parse dates for DOB and Start Date

If plan is selected:

Create RegularMember object

Set its attributes

Add to member list

Show success message

Clear fields

Else:

Show error for invalid plan

END FUNCTION

- Add Premium Member

#### START FUNCTION

METHOD addPremiumMember:

- Get input values from form fields
- Validate fields (including trainer's name)
- Parse dates
- Create PremiumMember object
- Set attributes
- Add to member list
- Show success message
- Clear fields

END FUNCTION

- Activate Membership

#### START FUNCTION

METHOD activateMembership:

- Get member ID
- Find member by ID
- If found:
  - Set status to Active
  - Show confirmation
- Else:

- Show error

END FUNCTION

- Deactive Membership

#### START FUNCTION

METHOD deactivateMembership:

- Get member ID
- Find member by ID
- If found:
  - Set status to Inactive
  - Show confirmation
- Else:

- Show error

END FUNCTION

- Mark Attendance

START FUNCTION

METHOD markAttendance:

Get member ID

Find member by ID

If found:

    Increase attendance count

    Show attendance confirmation

Else:

    Show error

END FUNCTION

- Upgrade Plan

START FUNCTION

METHOD upgradePlan:

Get member ID

Find member

If RegularMember:

    If BASIC -> upgrade to STANDARD

    If STANDARD -> upgrade to PREMIUM

    If PREMIUM -> show max plan reached

    Update displayed price

Else:

    Show error

END FUNCTION

- Calculate Discount

START FUNCTION

METHOD calculateDiscount:

Get member ID

Find member

If RegularMember:

    Calculate discount based on attendance

```
        Display in discount field
    Else:
        Show message - not applicable
END FUNCTION
```

- Revert Regular Member

```
START FUNCTION
METHOD revertRegularMember:
    Get member ID
    Find member
    If RegularMember:
        Revert to BASIC plan
        Update price field
        Show confirmation
    Else:
        Show error
END FUNCTION
```

- Revert Premium Member

```
START FUNCTION
METHOD revertPremiumMember:
    Get member ID
    Find member
    If PremiumMember:
        Clear trainer name
        Show confirmation
    Else:
        Show error
END FUNCTION
```

- Pay Due Amount

```
START FUNCTION
METHOD payDueAmount:
    Get member ID
```



```

Find member
If RegularMember:
    Calculate due = price - paid
    If due > 0:
        Add due to paid
        Show confirmation
    Else:
        Show message - no due
Else:
    Show error
END FUNCTION

```

- Display All Members

```

START FUNCTION
METHOD clearFields:
    Reset all input fields to blank
    Reset combo boxes and radio buttons
    Set discount to 0
END FUNCTION

```

- Save to file

```

START FUNCTION
METHOD saveToFile:
    Open file writer
    For each member:
        Write member.toFileString() to file
    Close writer
    Show confirmation
END FUNCTION

```

- Read from File

```

START FUNCTION
METHOD readFromFile:
    If file exists:
        Open file
        For each line:

```

```
        Convert to member using fromFileString()
        Add to list
        Show success message
    Else:
        Show error - no file
END FUNCTION
```

- Find Member by ID

```
START FUNCTION
METHOD findMemberById(id):
    For each member in list:
        If ID matches (case-insensitive):
            Return member
    Return null
END FUNCTION
```

## Method Description

### 1. Add Regular Member ()

It adds new Regular Member to the system by collecting user input from GUI and storing it in the member list.

```
addRegularMember();
```

Result: Adds Regular Members Like:

ID: 123,

Name: Alex

Plan: Regular

Start Date: 2025-05-01

## 2. Add Premium Member ()

It adds a Premium Member using the form data with premium specific details like trainer and higher pricing.

```
addPremiumMember();
```

Result: Adds a Premium Member like:

ID: 333

Name: Sam

Plan: Premium

Trainer: John

## 3. Activate Membership ()

It activates an inactive member based on ID.

```
activateMembership();
```

Result: Activates member

Membership for ID: 123 is activated successfully.

## 4. Deactivate Membership ()

It Deactivates a member and records the reason.

```
deactivateMembership();
```

Result: Deactivates

Membership for ID: 123 is deactivated. Reason: Moving to another City.

#### 5. Mark Attendance ()

Marks daily attendance for a member.

```
markAttendance();
```

Result:

Attendance marked for ID: 333 on 2025-05-06.

#### 6. Upgrades Plan ()

It converts a Regular Member into a Premium Member, keeping original data.

```
upgradePlan();
```

Result: Member ID: 123 is upgraded to Premium Plan.

#### 7. Calculate Discount ()

It applies discount logic, typically for Premium Members.

```
calculateDiscount();
```

Result:

Discount of 15% applied for ID: 456

#### 8. Revert Regular Member ()

It changes from Premium Member back to Regular.

```
revertRegularMember();
```

Result:

ID: 456 reverted to Regular Membership.

#### 9. Revert Premium Member ()

It cancels a revert and returns the member to Premium.

```
revertPremiumMember();
```

Result:

ID: 456 restored to Premium Membership.

#### 10. Pay Due Amount ()

It calculates and processes due payments.

```
payDueAmount();
```

Result:

Due amount paid for ID: 567. Receipt generated.

#### 11. Display Members ()

It displays all stored members in a readable format.

```
displayMembers();
```

Result:

ID: 123

Name: Alex

Status: Active

ID: 333

Name: Sam

Status Inactive

## 12. Clear Fields ()

It clears all input fields in the GUI form.

```
clearFields();
```

Result:

All the form fields are reset to empty.

## 13. Save To File ()

It saves all current members to a File.

```
saveToFile();
```

Result:

File "members.txt" created/updated with all members details.

#### 14. Read From File ()

It loads member data from a saved file.

```
readFromFile();
```

Result:

Populates member list with data from "members.txt".

#### 15. Find Member by Id (String Id)

It searches the list for a member with a specific ID.

```
GymMember m = findMemberById("R101");
```

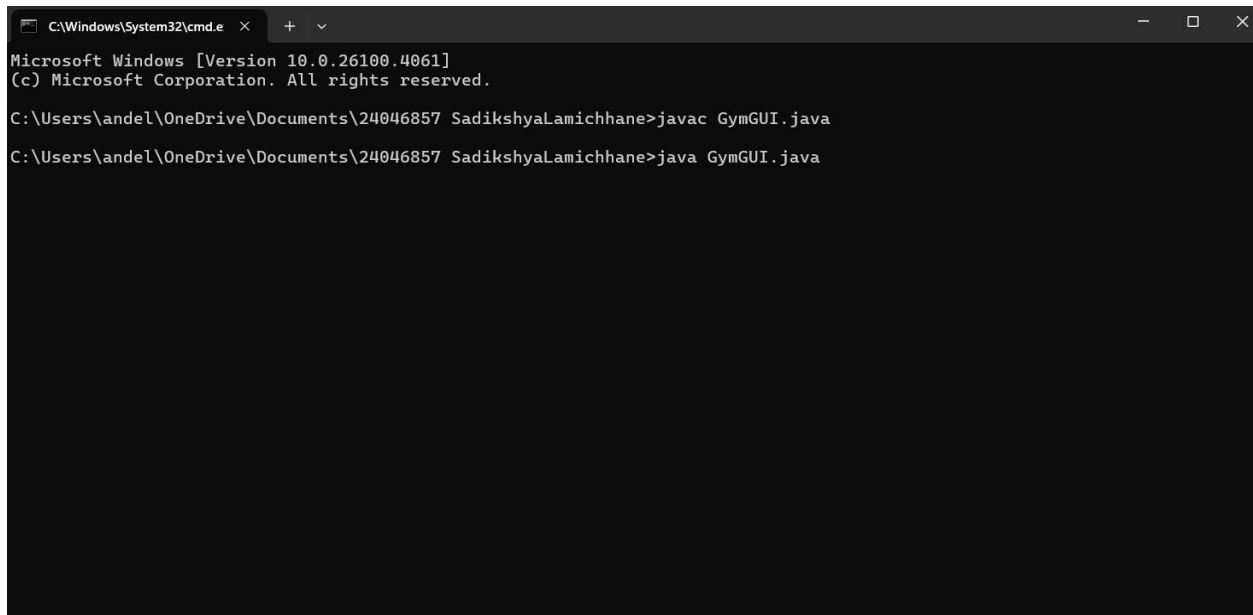
Result: Returns the Gym Member object if found, else null.

## Testing

### Test-1 Compile and Run program using Command Prompt/ Terminal

|                 |                                                                                                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objective       | To compile and run program using command prompt/ Terminal.                                                                                                                                               |
| Action          | The file manager was opened, and the file directory was changed to where the file is kept the command prompt and javac and the file name was written then compiled then finally the java file was opened |
| Expected Output | The java program should open.                                                                                                                                                                            |
| Actual Output   | The java program was open.                                                                                                                                                                               |
| Results         | The test was successful.                                                                                                                                                                                 |

Table 1 Test 1



```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [Version 10.0.26100.4061]
(c) Microsoft Corporation. All rights reserved.

C:\Users\andel\OneDrive\Documents\24046857 SadikshyaLamichhane>javac GymGUI.java
C:\Users\andel\OneDrive\Documents\24046857 SadikshyaLamichhane>java GymGUI.java
```



Image 13 test 1

islingtoncollegedunp-my.share Gym Management System

Member Information

Member ID: \_\_\_\_\_

Name: \_\_\_\_\_

Location: \_\_\_\_\_

Phone: \_\_\_\_\_

Email: \_\_\_\_\_

Gender: ☐ Male ☐ Female

Date of Birth: 1 01 1920

Membership Start Date: 1 01 1920

Referral Source: \_\_\_\_\_

Plan: BASIC

Regular Plan Price: \_\_\_\_\_

Premium Plan Price: \_\_\_\_\_

Paid Amount: \_\_\_\_\_

Discount Amount: \_\_\_\_\_

Removal Reason: \_\_\_\_\_

Trainer's Name: \_\_\_\_\_

Actions

|                       |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |

Actual C

Results

The test was successful.

Member Information

Member ID: \_\_\_\_\_

Name: \_\_\_\_\_

Location: \_\_\_\_\_

Phone: \_\_\_\_\_

Email: \_\_\_\_\_

Gender: ☐ Male ☐ Female

Date of Birth: 1 01 1920

Membership Start Date: 1 01 1920

Referral Source: \_\_\_\_\_

Plan: BASIC

Regular Plan Price: 2000.0

Premium Plan Price: 5000

Paid Amount: \_\_\_\_\_

Discount Amount: 0

Removal Reason: \_\_\_\_\_

Trainer's Name: \_\_\_\_\_

Actions

|                       |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |
| Save to File          | Read from File        |                       |

## Test-2

### Adding Regular Member

|                 |                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------|
| Objective       | To add a Regular Member                                                                         |
| Action          | All the required fields are filled, and “Add Regular Member” button was clicked.                |
| Expected Output | The Regular Member details should be added successfully with an appropriate message dialog box. |
| Actual Output   | The Regular Member details were added successfully with an appropriate message dialog box.      |
| Results         | The test was successful.                                                                        |

Table 2 Test 2

| Member Information     |                                                                    |
|------------------------|--------------------------------------------------------------------|
| Member ID:             | 1234                                                               |
| Name:                  | Sadikshya Lamichhane                                               |
| Location:              | Balaju                                                             |
| Phone:                 | 9803166206                                                         |
| Email:                 | sadikshya@gmail.com                                                |
| Gender:                | <input type="radio"/> Male <input checked="" type="radio"/> Female |
| Date of Birth:         | 7 03 2006                                                          |
| Membership Start Date: | 27 10 2019                                                         |
| Referral Source:       | samy                                                               |
| Plan:                  | STANDARD                                                           |
| Regular Plan Price:    | 4000.0                                                             |
| Premium Plan Price:    | 5000                                                               |
| Paid Amount:           | 4000                                                               |
| Discount Amount:       | 0                                                                  |
| Removal Reason:        | personal                                                           |
| Trainer's Name:        | Ham                                                                |

| Actions               |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |
| Save to File          | Read from File        |                       |

Image 14 Test 2

Member Information

Member ID:

1234

Name:

Sadikshya Lamichhane

Location:

Balaju

Phone:

9803166206

Email:

sadikshya@gmail.com

Gender:

☐ Male ☒ Female

Date of Birth:

7

03

2006

Membership Start Date:

27

10

2019

Re Message

Regular member added successfully!

OK

Paid Amount:

4000

Discount Amount:

0

Removal Reason:

personal

Trainer's Name:

Ham

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File





## Member List

MemberID: 1234  
Name: Sadikshya Lamichhane  
Phone9803166206  
Location:Balaju  
Email: sadikshya@gmail.com  
Gender: Female  
DOB:2006-03-07  
MembershipStartDate:2019-10-27  
ReferralSource:samy  
Membership Type: Regular  
Plan: STANDARD  
Paid Amount: 4000.0  
Calculated Price: 4000.0  
Attendance Count: 0  
Loyalty Points: 0  
Active Status: Active  
Membership Type: Regular

## Adding Premium Member

|                 |                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------|
| Objective       | Adding a Premium Member                                                                         |
| Action          | All the required fields are filled, and “Premium Member” button was clicked.                    |
| Expected Output | The Premium Member details should be added successfully with an appropriate message dialog box. |
| Actual Output   | The Premium Member details were added successfully with an appropriate message dialog box.      |
| Results         | The test was successful.                                                                        |

Table 3 Test 2

**Member Information**

|                        |                                                                    |
|------------------------|--------------------------------------------------------------------|
| Member ID:             | 333                                                                |
| Name:                  | Sam                                                                |
| Location:              | Kamalpokhari                                                       |
| Phone:                 | 97643456                                                           |
| Email:                 | sam@gmail.com                                                      |
| Gender:                | <input checked="" type="radio"/> Male <input type="radio"/> Female |
| Date of Birth:         | 12 09 2000                                                         |
| Membership Start Date: | 1 01 1920                                                          |
| Referral Source:       | Timmy                                                              |
| Plan:                  | BASIC                                                              |
| Regular Plan Price:    | 2000.0                                                             |
| Premium Plan Price:    | 5000                                                               |
| Paid Amount:           | 2000                                                               |
| Discount Amount:       | 0                                                                  |
| Removal Reason:        | personal                                                           |
| Trainer's Name:        | ham                                                                |

**Actions**

|                       |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |
| Save to File          | Read from File        |                       |

Image 15 Test 2

Member Information

Member ID:

333

Name:

Sam

Location:

Kamalpokhari

Phone:

97643456

Email:

sam@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

12

09

2000

Membership Start Date:

1

01

1920

Paid Amount:

2000

Discount Amount:

0

Removal Reason:

personal

Trainer's Name:

ham

Message

Premium member added successfully!

OK

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File

```

-----
ID: 333
Name: Sam
Email: sam@gmail.com
Phone:97643456
Plan: Premium
Start Date: 1920-01-01
Gender: Male
DOB:2000-09-12
MembershipStartDate:1920-01-01
ReferralSource:Timmy
Trainer: ham
Discount: 0.0
Paid Amount: $0.0%
Membership Type: Premium
-----

```

### Test- 3

#### Mark Attendance

| Objective       | To Mark Attendance                                                                            |
|-----------------|-----------------------------------------------------------------------------------------------|
| Action          | In the panel, the id of gym member needs to be added, and mark attendance button was clicked. |
| Expected Output | The attendance of the respective members is marked with the appropriate dialog box.           |
| Actual Output   | The attendance of the respective member is marked with the app                                |
| Result          | The test was successful.                                                                      |

Table 4 Test 3



**Member Information**

|                        |                                                                    |
|------------------------|--------------------------------------------------------------------|
| Member ID:             | 6223                                                               |
| Name:                  | Prashanga                                                          |
| Location:              | Lazimpat                                                           |
| Phone:                 | 97645678                                                           |
| Email:                 | pra@gmail.com                                                      |
| Gender:                | <input checked="" type="radio"/> Male <input type="radio"/> Female |
| Date of Birth:         | 30 06 2003                                                         |
| Membership Start Date: | 1 01 1920                                                          |
| Referral Source:       | Kritika                                                            |
| Plan:                  | STANDARD                                                           |
| Regular Plan Price:    | 4000.0                                                             |
| Premium Plan Price:    | 5000                                                               |
| Paid Amount:           | 4000                                                               |
| Discount Amount:       | 0                                                                  |
| Removal Reason:        | Personal                                                           |
| Trainer's Name:        | Dada                                                               |

**Actions**

|                       |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |
| Save to File          | Read from File        |                       |

Image 16 Test 3

Member Information

Member ID:

6223

Name:

Prashanga

Location:

Lazingpat

Phone:

97645678

Email:

pra@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

30

06

2003

Membership Start Date:

1

01

1920

Re Message

Regular member added successfully!

OK

Paid Amount:

4000

Discount Amount:

0

Removal Reason:

Personal

Trainer's Name:

Dada

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File

Member Information

Member ID:

6223

Name:

Prashanga

Location:

Lazimpat

Phone:

97845678

Email:

pra@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

30

06

2003

Membership Start Date:

1

01

1920

Re Message

×

Pl

i

Membership activated successfully!

OK

Paid Amount:

4000

Discount Amount:

0

Removal Reason:

Personal

Trainer's Name:

Dada

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File



Member Information

Member ID:

6223

Name:

Prashanga

Location:

Lazinpat

Phone:

97645678

Email:

pra@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

30

06

2003

Membership Start Date:

1

01

1920

Message

i

Attendance marked successfully for member ID: 6223

OK

Paid Amount:

4000

Discount Amount:

0

Removal Reason:

Personal

Trainer's Name:

Dada

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File

File Edit View

Attendance Marked Successfully for member ID: 6223 prashanga

## Test 4

### Calculate discount, pay due, revert members

|                 |                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------|
| Objective       | To Calculate Discount                                                                        |
| Action          | In the panel, all the member details should be added and then when we add to premium member. |
| Expected Output | The discount should be calculated.                                                           |
| Actual Output   | The discount was calculated for premium members.                                             |
| Result          | The test was successful.                                                                     |

Table 5 Test 4

**Member Information**

Member ID: 45

Name: Luffy

Location: island

Phone: 98765432

Email: luffy@gmail.com

Gender: ☒ Male ☐ Female

Date of Birth: 7 05 1925

Membership Start Date: 7 08 2017

Referral Source: zoro

Plan: PREMIUM

Regular Plan Price: 8000.0

Premium Plan Price: 5000

Paid Amount: 6000

Discount Amount: 0

Removal Reason:

Trainer's Name: San

**Actions**

|                       |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |
| Save to File          | Read from File        |                       |

Image 17 Test 4

Member Information

Member ID:

45

Name:

Luffy

Location:

island

Phone:

98765432

Email:

luffy@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

7

05

1925

Membership Start Date:

7

08

2017

Remove Message

Premium member added successfully!

OK

Paid Amount:

6000

Discount Amount:

0

Removal Reason:

Trainer's Name:

San

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File



Member Information

Member ID:

45

Name:

Luffy

Location:

island

Phone:

98765432

Email:

luffy@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

7

05

1925

Membership Start Date:

7

08

2017

Ref

Message

×

Pla

i

Discount calculated successfully!

Reg

OK

Pre

Paid Amount:

8000

Discount Amount:

0

Removal Reason:

Trainer's Name:

San

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File



|                 |                                                                                                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Objective       | To Pay Due                                                                                                                               |
| Action          | In the panel, all the member details should be added and then if we have not paid the full amount of of the membership we should pay it. |
| Expected Output | The payment due should be paid.                                                                                                          |
| Actual Output   | The payment due was paid.                                                                                                                |
| Result          | The test was successful.                                                                                                                 |

Table 6 Test 4

**Member Information**

Member ID: 777

Name: Zoro

Location: Antatica

Phone: 9876543

Email: zoro@gmail.com

Gender: ☒ Male ☐ Female

Date of Birth: 11 / 09 / 2000

Membership Start Date: 7 / 08 / 2016

Referral Source: san

Plan: BASIC

Regular Plan Price: 2000.0

Premium Plan Price: 5000

Paid Amount: 1000

Discount Amount: 0

Removal Reason:

Trainer's Name: luf

**Actions**

Add Regular Member   Add Premium Member   Activate Membership

Deactivate Membership   Mark Attendance   Upgrade Plan

Calculate Discount   Revert Regular Member   Revert Premium Member

Pay Due Amount   Display Members   Clear Fields

Save to File   Read from File

Image 18 Test 4

Member Information

Member ID:

777

Name:

Zoro

Location:

Antatica

Phone:

9876543

Email:

zoro@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

11

09

2000

Membership Start Date:

7

08

2016

Re Message

Regular member added successfully!

OK

Paid Amount:

1000

Discount Amount:

0

Removal Reason:

Trainer's Name:

Iuf

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File

Member Information

Member ID:

777

Name:

Zoro

Location:

Antatica

Phone:

9876543

Email:

zoro@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

11

09

2000

Membership Start Date:

7

08

2016

Re Message

×

Pla

i

Membership activated successfully!

Re

OK

Pro

Paid Amount:

1000

Discount Amount:

0

Removal Reason:

Trainer's Name:

luf

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File



Member Information

Member ID:

777

Name:

Zoro

Location:

Antatica

Phone:

9876543

Email:

zoro@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

11

09

2000

Membership Start Date:

7

08

2016

Message

Due amount of 1000.0 paid successfully.

OK

Paid Amount:

2000.0

Discount Amount:

0

Removal Reason:

Trainer's Name:

Iuf

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File

|                 |                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------|
| Objective       | To add Revert Member                                                                                                 |
| Action          | In the panel, all the member details should be added for Premium member and then we can revert it to Regular Member. |
| Expected Output | Premium Member should be reverted to Regular Member.                                                                 |
| Actual Output   | Premium Member was Reverted to Regular Member.                                                                       |
| Result          | The test was successful.                                                                                             |

Table 7 Test 4

**Member Information**

Member ID: 666

Name: Nami

Location: gold

Phone: 98765432

Email: Nami@gmail.com

Gender: ☐ Male ☒ Female

Date of Birth: 6 / 06 / 2001

Membership Start Date: 6 / 05 / 2013

Referral Source: Zo

Plan: BASIC

Regular Plan Price: 2000.0

Premium Plan Price: 5000

Paid Amount: 2000

Discount Amount: 0

Removal Reason:

Trainer's Name: zoro

**Actions**

Add Regular Member   Add Premium Member   Activate Membership

Deactivate Membership   Mark Attendance   Upgrade Plan

Calculate Discount   Revert Regular Member   Revert Premium Member

Pay Due Amount   Display Members   Clear Fields

Save to File   Read from File

Image 19 Test 4

Member Information

Member ID:

660

Name:

Nami

Location:

gold

Phone:

98765432

Email:

Nami@gmail.com

Gender:

☐ Male ☒ Female

Date of Birth:

6

06

2001

Membership Start Date:

6

05

2013

Message

Premium member added successfully!

OK

Paid Amount:

2000

Discount Amount:

0

Removal Reason:

Trainer's Name:

zoroo

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File



Member Information

Member ID:

660

Name:

Nami

Location:

gold

Phone:

98765432

Email:

Nami@gmail.com

Gender:

☐ Male ☒ Female

Date of Birth:

6

06

2001

Membership Start Date:

6

05

2013

Re Message

×

Pl

i

Premium member removed from list.

Re

OK

Pr

Paid Amount:

2000

Discount Amount:

0

Removal Reason:

Trainer's Name:

zoroo

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File



## Test 5

### Save and Read from File

|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| Objective       | Save and Read from File                                                             |
| Action          | In the panel, all the member details should be added, and we need to save the file. |
| Expected Output | The member details should be saved, and member details should be shown.             |
| Actual Output   | The Member details were saved and member details were shown.                        |
| Result          | The test was successful.                                                            |

Table 8 Test 5



Member Information

Member ID:

Name:

Location:

Phone:

Email:

Gender: ☐ Male ☐ Female

Date of Birth:

Membership Start Date:

Ref Information

Pla

Reg

Pre

Paid Amount:

Discount Amount:

Removal Reason:

Trainer's Name:

Actions

|                       |                       |                       |
|-----------------------|-----------------------|-----------------------|
| Add Regular Member    | Add Premium Member    | Activate Membership   |
| Deactivate Membership | Mark Attendance       | Upgrade Plan          |
| Calculate Discount    | Revert Regular Member | Revert Premium Member |
| Pay Due Amount        | Display Members       | Clear Fields          |
| Save to File          | Read from File        |                       |

No members to save!

OK

Image 20 Test 5

Member Information

Member ID:

527

Name:

Sanji

Location:

Ship

Phone:

34564567

Email:

sanji@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

19▼08▼1988▼

Membership Start Date:

1▼01▼1920▼

Re Message

×

PL

i

Regular member added successfully!

Re

OK

Pr

Paid Amount:

2000

Discount Amount:

0

Removal Reason:

Trainer's Name:

Food

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File

Member Information

Member ID:

527

Name:

Sanji

Location:

Ship

Phone:

34564567

Email:

sanji@gmail.com

Gender:

☒ Male ☐ Female

Date of Birth:

19

08

1988

Membership Start Date:

1

01

1920

Re Message

×

Pl

Re

Pr

Membership activated successfully!

OK

Paid Amount:

2000

Discount Amount:

0

Removal Reason:

Trainer's Name:

Food

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File



Member Information

Member ID:

527

Name:

Sanji

Location:

Ship

Phone:

34564567

Email:

sanji@gmail.com

Gender:

☒ Male
☐ Female

Date of Birth:

19

08

1988

Membership Start Date:

1

01

1920

Referral Source:

fdfgh

Plan:

BASIC

Regular Plan Price:

2000.0

Premium Plan Price:

5000

Paid Amount:

2000

Discount Amount:

0

Removal Reason:

Trainer's Name:

Food

Actions

Add Regular Member

Add Premium Member

Activate Membership

Deactivate Membership

Mark Attendance

Upgrade Plan

Calculate Discount

Revert Regular Member

Revert Premium Member

Pay Due Amount

Display Members

Clear Fields

Save to File

Read from File

Member List

MemberID: 527  
Name: Sanji  
Phone34564567  
Location:Ship  
Email: sanji@gmail.com  
Gender: Male  
DOB:1988-08-19  
MembershipStartDate:1920-01-01  
ReferralSource: fdfgh  
Membership Type: Regular  
Plan: BASIC  
Paid Amount: 2000.0  
Calculated Price: 2000.0  
Attendance Count: 1  
Loyalty Points: 1  
Active Status: Active  
Membership Type: Regular  
-----

# Error Detection and Correction

## Syntax Error 1

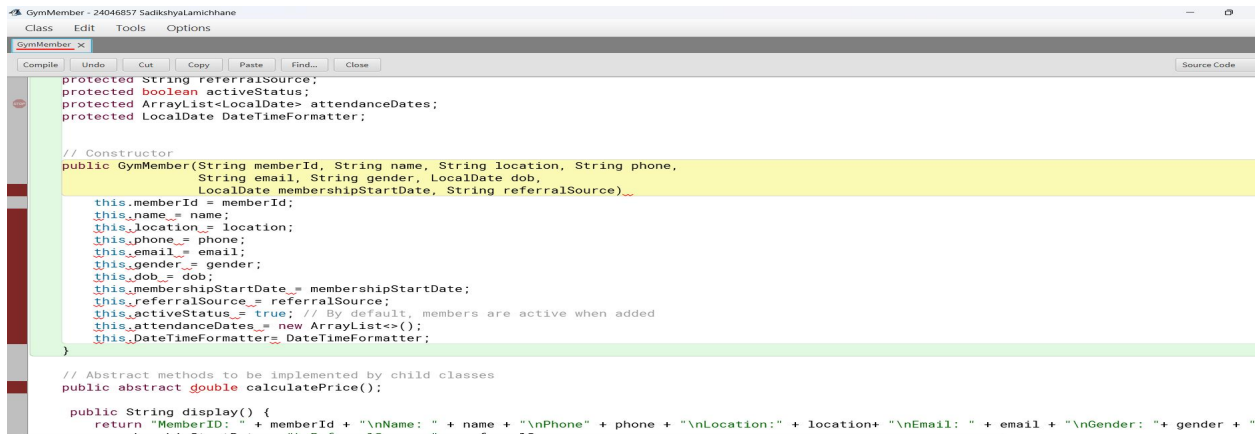


Image 21 Syntax Error

Cause of error:

Here, in this program there is an unmatched Braces {}, which leads to syntax Error. This error is followed by public Gym Member line in constructor.

Correction

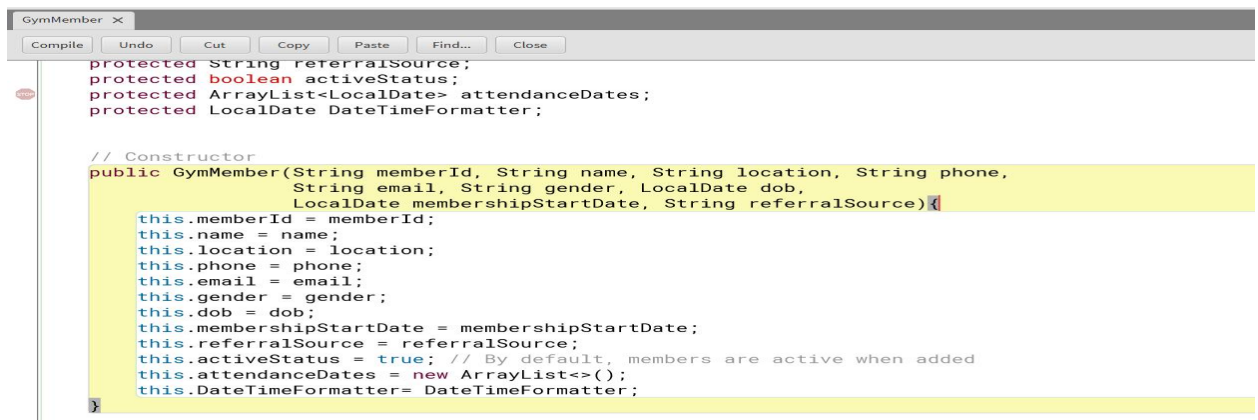


Image 22 Syntax error correction

Corrected form:

Now when we add the missing braces { in its place and compile the program it starts to work without showing any errors.

## Syntax Error 2

```
+ membershipStartDate + "\nReferralSource:" + referralSo
}

// Method to activate membership
public void activateMembership{ } {
    if (!activeStatus) {
        activeStatus = true;
        System.out.println("Membership activated for member:
    } else {
        System.out.println("Membership is already active for
    }
}
```

Cause of error:

Here, an error has occurred in this class due to the incorrect method of declaration of method. A () is needed in place of {} to make the program run.

```
// Method to activate membership
public void activateMembership() {
    if (!activeStatus) {
        activeStatus = true;
        System.out.println("Membership activated for member: " + name);
    } else {
        System.out.println("Membership is already active for member: " + name);
    }
}
```

Corrected program:

Here we have removed the unessential {} and replaced it with () which is the correct method of declaration.

## Runtime Error

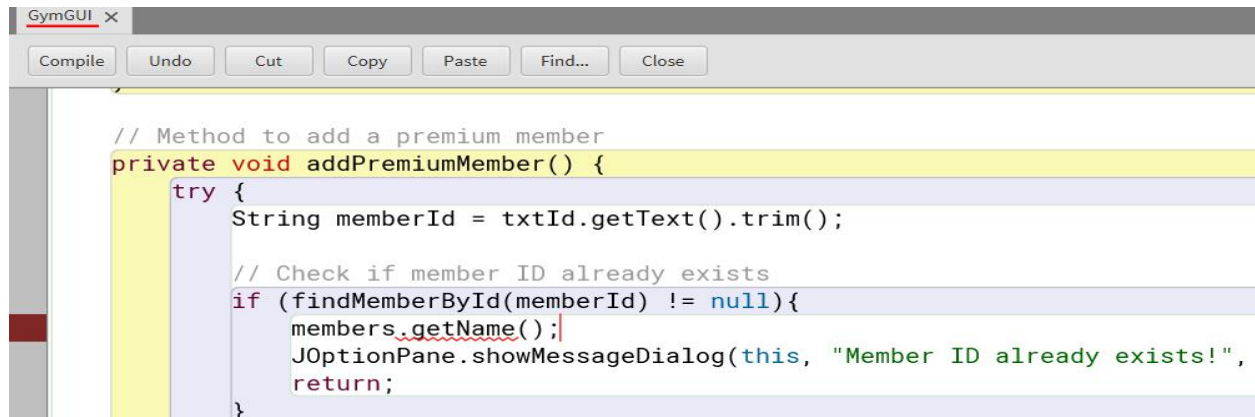


Image 23 Run time error

Cause of error:

Here in this program, the program crashes while trying to access something that doesn't exist. In member. get name (); the member is null which causes the run time error.

Correction:



Image 24 run time error correction

Corrected Program:

When we remove the null member and the part which does not exist in the program and compile the program will run without showing any error.

## Logical Error

```
txtRegularPrice.setText(String.valueOf(RegularMember.Plan.BASIC.getPr  
  
// Initialize combo boxes for DOB  
String[] days = new String[31];  
for (int i = 0; i < 31; i++) {  
    days[i] = String.valueOf(i - 1);  
}  
cbDay = new JComboBox<>(days);
```

Image 25 logical error

Cause of error;

Here in this program, the code has skipped day 1. the array here is containing “0” to “30” instead of “1” to “31”. There is no such thing as day 0 in a calendar, which is a logical error!

Correction:

```
txtRegularPrice.setText(String.valueOf(RegularMember.Plan.BASIC.getPr  
  
// Initialize combo boxes for DOB  
String[] days = new String[31];  
for (int i = 0; i < 31; i++) {  
    days[i] = String.valueOf(i + 1);  
}  
cbDay = new JComboBox<>(days);
```

Image 26 logical error correction

Corrected Program:

The program is corrected and now, this correctly fills the combo box with “1” to “31” (Valid Day Numbers).



# Conclusion

## Evaluation of Work

- Reflection and Learnings

Working on this coursework has been a valuable learning experience. It allowed me to apply theoretical knowledge and practical tasks, depending on my understanding of key concepts. I learned the importance of planning, testing, and refining work, especially when dealing with real-world scenarios. It also enhanced my technical skills, time management, and attention to detail.

Developing the Gym Member Management System using Java System using Java Swing has been a highly educational experience. I get deeper understanding of inheritance by managing Regular and Premium Member through a shared superclass and learned how to implement interactive GUI components that respond to user input.

- Difficulties Faced

One of the main challenges was managing multiple data inputs across various member types while ensuring the program remained user-friendly and efficient. Implementation features like plan upgrades, discount calculation, attendance tracking and membership activation/ deactivation required careful logic and error checking. GUI layout management using Java Swing was another difficulty, especially in maintaining alignment and functionality for all buttons and fields. Additionally, reading from and writing to files while maintaining data consistency posed some problems.

- How Difficulties Were Overcome

I overcame these challenges by breaking down each feature into smaller, manageable steps and thoroughly testing them as I went. I went through a combination of research, trial, error, and perseverance. Referring to Java Swing documentation and examples helped me resolve GUI alignment issues. I referred to official documentation, online tutorials, and sought feedback.

For file handling and logic implementation, I used systematic debugging techniques and added validation to ensure reliability. I also reused and refactored code of efficiency, especially for common tasks across both member types. This imitative process significantly enhanced both the quality of the application and my confidence in java development.

Overall, The Gym Management System project has been a compressive and practical application of object-oriented programming and GUI development in Java. It brought together multiple core concepts such as inheritance, data validation, file handling, and event-driven programming into a functional and user-friendly system for managing Regular and Premium gym members.

This coursework, I not only improved my skills in java but also developed a deeper understanding of how to plan, structure and trouble =shoots real-world applications. From designing the interface to implementing features like membership tracking, attendance, and payment management, every stage contributed to building my confidence and problem-solving abilities.

## References

1. "Java: A Beginner's Guide" by Herbert Schildt
2. "Java Projects: Learn the Art of Building Modern Java Applications" by Peter Verhas
3. "Data Structures and Algorithms in Java" by Robert Lafore
4. Oracle's Java Tutorials
5. Building Java Programs: A Back to Basics Approach" by Stuart Reges & Marty Stepp

## APPENDIX

### GymMember

```
import java.time.LocalDate;
```

```
import java.util.ArrayList;
```

```
/**
```

```
 * Write a description of class GymMember here.
```

```
 *
```

```
 * @author (your name)
```

```
 * @version (a version number or a date)
```

```
 */
```

```
public abstract class GymMember
```

```
{
```

```
    // Common attributes for all gym members
```

```
    protected String memberId;
```

```
    protected String name;
```

```
    protected String location;
```

```
    protected String phone;
```

```
    protected String email;
```

```
    protected String gender;
```

```
    protected LocalDate dob;
```

```

protected LocalDate membershipStartDate;
protected String referralSource;
protected boolean activeStatus;
protected ArrayList<LocalDate> attendanceDates;
protected LocalDate DateTimeFormatter;

// Constructor
public GymMember(String memberId, String name, String location, String phone,
                  String email, String gender, LocalDate dob,
                  LocalDate membershipStartDate, String referralSource){
    this.memberId = memberId;
    this.name = name;
    this.location = location;
    this.phone = phone;
    this.email = email;
    this.gender = gender;
    this.dob = dob;
    this.membershipStartDate = membershipStartDate;
    this.referralSource = referralSource;
    this.activeStatus = true; // By default, members are active when added
    this.attendanceDates = new ArrayList<>();
    this.DateTimeFormatter= DateTimeFormatter;
}

// Abstract methods to be implemented by child classes

```

```

public abstract double calculatePrice();

public String display() {
    return "MemberID: " + memberId + "\nName: " + name + "\nPhone" + phone +
"\nLocation:" + location+ "\nEmail: " + email + "\nGender: "+ gender + "\nDOB:" + dob +
"\nMembershipStartDate:"
    + membershipStartDate + "\nReferralSource:" + referralSource;
}

// Method to activate membership
public void activateMembership() {
    if (!activeStatus) {
        activeStatus = true;
        System.out.println("Membership activated for member: " + name);
    } else {
        System.out.println("Membership is already active for member: " + name);
    }
}

protected String removalReason;

public void setRemovalReason(String reason) {
    this.removalReason = reason;
}

public String getRemovalReason() {
    return removalReason;
}

```

```
}
```

```
public boolean isActiveStatus() {  
    return activeStatus;  
}
```

```
public void setActiveStatus(boolean activeStatus) {  
    this.activeStatus = activeStatus;  
}
```

```
// Method to deactivate membership
```

```
public void deactivateMembership() {  
    if (activeStatus) {  
        activeStatus = false;  
        System.out.println("Membership deactivated for member: " + name);  
    } else {  
        System.out.println("Membership is already inactive for member: " + name);  
    }  
}
```

```
// Method to mark attendance
```

```
public void markAttendance() {  
    if (activeStatus) {  
        LocalDate today = LocalDate.now();  
        attendanceDates.add(today);  
        System.out.println("Attendance marked for " + name + " on " + today);  
    }  
}
```



```
    } else {  
        System.out.println("Cannot mark attendance for inactive member: " + name);  
    }  
}
```

// Getters and Setters

```
public String getMemberId() {  
    return memberId;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public String getLocation() {  
    return location;  
}
```

```
public String getPhone() {  
    return phone;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
public String getGender() {  
    return gender;  
}
```

```
public LocalDate getDob() {  
    return dob;  
}
```

```
public LocalDate getMembershipStartDate() {  
    return membershipStartDate;  
}
```

```
public String getReferralSource() {  
    return referralSource;  
}
```

```
public ArrayList<LocalDate> getAttendanceDates() {  
    return attendanceDates;  
}
```

```
public int getAttendanceCount() {  
    return attendanceDates.size();  
}
```

```
@Override
```

```
public String toString() {
```

```
        return "ID: " + memberId + "\nName: " + name + "\nMembership Type: Regular";
    }
}
```

## RegularMember

```
import java.time.LocalDate;
```

```
public class RegularMember extends GymMember {
```

```
    public enum Plan {
```

```
        BASIC(2000),
```

```
        STANDARD(4000),
```

```
        PREMIUM(8000);
```

```
        private final double price;
```

```
        Plan(double price) {
```

```
            this.price = price;
```

```
        }
```

```
        public double getPrice() {
```

```
            return price;
```

```
        }
```

```
    }
```

```
    private Plan plan;
```

```

private double paidAmount;
private int loyaltyPoints;

private static final double DISCOUNT_RATE = 0.05;

// Constructor
public RegularMember(String memberId, String name, String location, String phone,
                    String email, String gender, LocalDate dob,
                    LocalDate membershipStartDate, String referralSource,
                    Plan plan, double paidAmount) {
    super(memberId, name, location, phone, email, gender, dob,
membershipStartDate, referralSource);
    this.plan = plan;
    this.paidAmount = paidAmount;
    this.loyaltyPoints = 0;
}

public void revertMember(String reason) {
    this.setPlan(Plan.BASIC);
    this.setPaidAmount(0);
    this.setLoyaltyPoints(0);
    this.attendanceDates.clear();
    this.setActiveStatus(false);
    this.setRemovalReason(reason); // Only if removalReason is a field in GymMember
    System.out.println("Regular member " + name + " has been reverted for reason: " +
reason);
}

```

```

@Override
public double calculatePrice() {
    double basePrice = plan.getPrice();

    if (getAttendanceCount() > 10) {
        basePrice -= basePrice * DISCOUNT_RATE;
    }

    return basePrice;
}

```

```

@Override
public void markAttendance() {
    if (activeStatus) {
        LocalDate today = LocalDate.now();
        if (!attendanceDates.contains(today)) {
            attendanceDates.add(today);
            loyaltyPoints++;

            if (getAttendanceCount() == 11) {
                loyaltyPoints += 5;
            }

            System.out.println("Attendance marked for " + name + " on " + today);
            System.out.println("Loyalty Points: " + loyaltyPoints);
        }
    }
}

```

```

    } else {
        System.out.println("Attendance already marked today for " + name);
    }
} else {
    System.out.println("Cannot mark attendance for inactive member: " + name);
}
}

```

```

public boolean upgradePlan(Plan newPlan) {
    if (newPlan.ordinal() > this.plan.ordinal()) {
        this.plan = newPlan;
        this.paidAmount = newPlan.getPrice();
        System.out.println("Plan upgraded to: " + newPlan.name());
        return true;
    } else {
        System.out.println("Selected plan is not higher than current.");
        return false;
    }
}

```

// Getters and Setters

```

public Plan getPlan() {
    return plan;
}

```

```

public void setPlan(Plan plan) {

```

```

        this.plan = plan;
    }

    public double getPaidAmount() {
        return paidAmount;
    }

    public void setPaidAmount(double paidAmount) {
        this.paidAmount = paidAmount;
    }

    public int getLoyaltyPoints() {
        return loyaltyPoints;
    }

    public void setLoyaltyPoints(int loyaltyPoints) {
        this.loyaltyPoints = loyaltyPoints;
    }

    @Override
    public String display() {
        return super.display()
            + "\nMembership Type: Regular"
            + "\nPlan: " + plan.name()
            + "\nPaid Amount: " + paidAmount
            + "\nCalculated Price: " + calculatePrice()
            + "\nAttendance Count: " + getAttendanceCount()
            + "\nLoyalty Points: " + loyaltyPoints
    }

```

```

        + "\nActive Status: " + (activeStatus ? "Active" : "Inactive");
    }
}

```

## PremiumMember

```
import java.time.LocalDate;
```

```

public class PremiumMember extends GymMember
{
    // Premium member specific attributes
    private static final double PREMIUM_PRICE = 5000;
    private String personalTrainer;
    private double paidAmount;
    private double discountAmount;

    // Constructor
    public PremiumMember(String memberId, String name, String location, String phone,
        String email, String gender, LocalDate dob,
        LocalDate membershipStartDate, String referralSource,
        String personalTrainer) {
        super(memberId, name, location, phone, email, gender, dob,
membershipStartDate, referralSource);
        this.personalTrainer = personalTrainer;
        this.paidAmount = 0;
        this.discountAmount = 0;
    }
}

```



```

// Method to calculate discount based on attendance
public void calculateDiscount() {

    if (isActiveStatus()) {
        int attendanceCount = getAttendanceCount();
        if (attendanceCount==1 ) {
            discountAmount = 0.15 * PREMIUM_PRICE; // 15% discount
        } else if (attendanceCount > 10) {
            discountAmount = 0.05 * PREMIUM_PRICE; // 5% discount
        } else {
            discountAmount = 0;
        }

        System.out.println("Discount calculated for " + getName() + ": " +
discountAmount);
    } else {
        System.out.println("Cannot calculate discount for inactive member: " +
getName());
    }
}

// Method to pay due amount
public void payDueAmount(double amount) {
    if (isActiveStatus()) {
        if (amount > 0) {
            paidAmount += amount;
            System.out.println("Payment of " + amount + " received from " + getName());
        }
    }
}

```

```

        } else {
            System.out.println("Invalid payment amount.");
        }
    } else {
        System.out.println("Cannot receive payment for inactive member: " +
getName());
    }
}

// Method to revert member
public void revertMember(String reason) {
    deactivateMembership();

    System.out.println("Premium member " + getName() + " reverted due to: " +
reason);
}

// Method to calculate price
@Override
public double calculatePrice() {
    return PREMIUM_PRICE - discountAmount;
}

// Check if full payment is made
public boolean isFullPayment() {
    return paidAmount >= calculatePrice();
}

```

```

// Getters
public String getPersonalTrainer() {
    return personalTrainer;
}

public double getPaidAmount() {
    return paidAmount;
}

public double getDiscountAmount() {
    return discountAmount;
}

public double getRemainingAmount() {
    return calculatePrice() - paidAmount;
}

// String representation of the object
@Override
public String toString() {
    return super.toString() + String.format(" %-10s %-15.2f %-15.2f %-15.2f",
        personalTrainer, calculatePrice(), discountAmount, paidAmount);
}

@Override
public String display() {
    return "ID: " + memberId
        + "\nName: " + name + "\nEmail: " + email + "\nPhone: " + phone +
        "\nPlan: Premium\nStart Date: " + membershipStartDate + "\nGender: " + gender
        + "\nDOB: " + dob + "\nMembershipStartDate: "
        + membershipStartDate + "\nReferralSource: " + referralSource +

```

```
        "\nTrainer: " + personalTrainer + "\nDiscount: " + discountAmount + "\nPaid  
Amount: $" + paidAmount + "%";
```

```
}
```

```
}
```

## GymGUI

```
import java.util.ArrayList;
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.ActionEvent;
```

```
import java.awt.event.ActionListener;
```

```
import java.time.LocalDate;
```

```
import java.time.format.DateTimeFormatter;
```

```
import java.io.FileWriter;
```

```
import java.io.IOException;
```

```
import java.io.File;
```

```
import java.io.BufferedReader;
```

```
import java.io.FileReader;
```

```
public class GymGUI extends JFrame {
```

```
    // ArrayList to store gym members
```

```
    private ArrayList<GymMember> members;
```

```
    // GUI components
```

```
    private JTextField txtId, txtName, txtLocation, txtPhone, txtEmail;
```

```
    private JComboBox<String> cbDay, cbMonth, cbYear;
```

```
    private JComboBox<String> cbStartDateDay, cbStartDateMonth, cbStartDateYear;
```

```
    private JTextField txtReferral, txtPaidAmount, txtRemovalReason, txtTrainer;
```

```
private JTextField txtRegularPrice, txtPremiumPrice, txtDiscountAmount;  
private JRadioButton rbMale, rbFemale;  
private ButtonGroup bgGender;  
private JComboBox<RegularMember.Plan> cbPlan;
```

```
// Constructor
```

```
public GymGUI() {
```

```
    members = new ArrayList<>();
```

```
    // Set up the frame
```

```
    setTitle("Gym Management System");
```

```
    setSize(900, 650);
```

```
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
    setLocationRelativeTo(null);
```

```
    // Set background color for the frame
```

```
    ImageIcon backgroundImage = new  
    ImageIcon("C:/Users/andel/Downloads/download (1).jpeg"); // Use your actual image  
    path
```

```
    JLabel backgroundLabel = new JLabel(backgroundImage);
```

```
    backgroundLabel.setLayout(new BorderLayout());
```

```
    setContentPane(backgroundLabel);
```

```
    JPanel inputPanel = new JPanel(new GridLayout(30, 30));
```

```

inputPanel.setBackground(new Color(0, 0, 0));

// Create main panel with GridBagLayout
JPanel mainPanel = new JPanel(new GridBagLayout());
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(5, 5, 5, 5);

// Create form panel
JPanel formPanel = createFormPanel();

// Create button panel
JPanel buttonPanel = createButtonPanel();

// Add panels to main panel
gbc.gridx = 0;
gbc.gridy = 0;
gbc.fill = GridBagConstraints.BOTH;
mainPanel.add(formPanel, gbc);

gbc.gridy = 1;
mainPanel.add(buttonPanel, gbc);

// Add main panel to frame
add(mainPanel);
mainPanel.setOpaque(false);
formPanel.setOpaque(false);

```

```

        buttonPanel.setOpaque(false);

        setVisible(true);
    }

    // Create form panel with input fields
    private JPanel createFormPanel() {
        JPanel panel = new JPanel(new GridBagLayout());
        panel.setOpaque(false);
        panel.setBorder(BorderFactory.createTitledBorder("Member Information"));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.anchor = GridBagConstraints.WEST;

        // Initialize input fields
        txtId = new JTextField(15);
        txtName = new JTextField(15);
        txtLocation = new JTextField(15);
        txtPhone = new JTextField(15);
        txtEmail = new JTextField(15);
        txtReferral = new JTextField(15);
        txtPaidAmount = new JTextField(15);
        txtRemovalReason = new JTextField(15);
        txtTrainer = new JTextField(15);
    }

```

```

// Initialize non-editable fields
txtRegularPrice = new JTextField(15);
txtRegularPrice.setEditable(false);
txtPremiumPrice = new JTextField("5000");
txtPremiumPrice.setEditable(false);
txtDiscountAmount = new JTextField("0");
txtDiscountAmount.setEditable(false);

// Initialize radio buttons for gender
rbMale = new JRadioButton("Male");
rbFemale = new JRadioButton("Female");
bgGender = new ButtonGroup();
bgGender.add(rbMale);
bgGender.add(rbFemale);
JPanel genderPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
genderPanel.add(rbMale);
genderPanel.add(rbFemale);

// Initialize combo box for plan
cbPlan = new JComboBox<>(RegularMember.Plan.values());
cbPlan.addActionListener(e -> {
    RegularMember.Plan selectedPlan = (RegularMember.Plan)
cbPlan.getSelectedItem();
    txtRegularPrice.setText(String.valueOf(selectedPlan.getPrice()));
    panel.setBackground(new Color(255, 255, 204));
});

```



```

// Set initial value for regular price
txtRegularPrice.setText(String.valueOf(RegularMember.Plan.BASIC.getPrice()));

// Initialize combo boxes for DOB
String[] days = new String[31];
for (int i = 0; i < 31; i++) {
    days[i] = String.valueOf(i + 1);
}
cbDay = new JComboBox<>(days);

String[] months = {"01", "02", "03", "04", "05", "06", "07", "08", "09", "10", "11", "12"};
cbMonth = new JComboBox<>(months);

String[] years = new String[100];
for (int i = 0; i < 100; i++) {
    years[i] = String.valueOf(1920 + i);
}
cbYear = new JComboBox<>(years);

// Initialize combo boxes for Membership Start Date
cbStartDateDay = new JComboBox<>(days);
cbStartDateMonth = new JComboBox<>(months);
cbStartDateYear = new JComboBox<>(years);

// Add components to panel with labels
int row = 0;

```

```
addLabelAndField(panel, "Member ID:", txtId, gbc, row++);
addLabelAndField(panel, "Name:", txtName, gbc, row++);
addLabelAndField(panel, "Location:", txtLocation, gbc, row++);
addLabelAndField(panel, "Phone:", txtPhone, gbc, row++);
addLabelAndField(panel, "Email:", txtEmail, gbc, row++);
```

```
// Add gender radio buttons
gbc.gridx = 0;
gbc.gridy = row;
panel.add(new JLabel("Gender:"), gbc);
gbc.gridx = 1;
panel.add(genderPanel, gbc);
row++;
```

```
// Add DOB combo boxes
gbc.gridx = 0;
gbc.gridy = row;
panel.add(new JLabel("Date of Birth:"), gbc);
gbc.gridx = 1;
JPanel dobPanel = new JPanel();
dobPanel.add(cbDay);
dobPanel.add(cbMonth);
dobPanel.add(cbYear);
panel.add(dobPanel, gbc);
row++;
```

```

// Add Membership Start Date combo boxes
gbc.gridx = 0;
gbc.gridy = row;
panel.add(new JLabel("Membership Start Date:"), gbc);
gbc.gridx = 1;
JPanel startDatePanel = new JPanel();
startDatePanel.add(cbStartDateDay);
startDatePanel.add(cbStartDateMonth);
startDatePanel.add(cbStartDateYear);
panel.add(startDatePanel, gbc);
row++;

addLabelAndField(panel, "Referral Source:", txtReferral, gbc, row++);
addLabelAndField(panel, "Plan:", cbPlan, gbc, row++);
addLabelAndField(panel, "Regular Plan Price:", txtRegularPrice, gbc, row++);
addLabelAndField(panel, "Premium Plan Price:", txtPremiumPrice, gbc, row++);
addLabelAndField(panel, "Paid Amount:", txtPaidAmount, gbc, row++);
addLabelAndField(panel, "Discount Amount:", txtDiscountAmount, gbc, row++);
addLabelAndField(panel, "Removal Reason:", txtRemovalReason, gbc, row++);
addLabelAndField(panel, "Trainer's Name:", txtTrainer, gbc, row++);

return panel;
}

// Helper method to add label and field to panel

```

```

private void addLabelAndField(JPanel panel, String labelText, JTextField field,
GridBagConstraints gbc, int row) {
    gbc.gridx = 0;
    gbc.gridy = row;
    panel.add(new JLabel(labelText), gbc);
    gbc.gridx = 1;
    panel.add(field, gbc);
}

```

```

private void addLabelAndField(JPanel panel, String labelText, JComboBox<?>
comboBox, GridBagConstraints gbc, int row) {
    gbc.gridx = 0;
    gbc.gridy = row;
    panel.add(new JLabel(labelText), gbc);
    gbc.gridx = 1;
    panel.add(comboBox, gbc);
}

```

// Create button panel with all action buttons

```

private JPanel createButtonPanel() {
    JPanel panel = new JPanel(new GridLayout(0, 3, 10, 10));
    panel.setOpaque(false);
    panel.setBorder(BorderFactory.createTitledBorder("Actions"));

    // Create buttons
    JButton btnAddRegular = new JButton("Add Regular Member");
    JButton btnAddPremium = new JButton("Add Premium Member");
}

```

```

JButton btnActivate = new JButton("Activate Membership");
JButton btnDeactivate = new JButton("Deactivate Membership");
JButton btnMarkAttendance = new JButton("Mark Attendance");
JButton btnUpgradePlan = new JButton("Upgrade Plan");
JButton btnCalculateDiscount = new JButton("Calculate Discount");
JButton btnRevertRegular = new JButton("Revert Regular Member");
JButton btnRevertPremium = new JButton("Revert Premium Member");
JButton btnPayDue = new JButton("Pay Due Amount");
JButton btnDisplay = new JButton("Display Members");
JButton btnClear = new JButton("Clear Fields");
JButton btnSaveToFile = new JButton("Save to File");
JButton btnReadFromFile = new JButton("Read from File");

// Add action listeners to buttons
btnAddRegular.addActionListener(e -> addRegularMember());
btnAddPremium.addActionListener(e -> addPremiumMember());
btnActivate.addActionListener(e -> activateMembership());
btnDeactivate.addActionListener(e -> deactivateMembership());
btnMarkAttendance.addActionListener(e -> markAttendance());
btnUpgradePlan.addActionListener(e -> upgradePlan());
btnCalculateDiscount.addActionListener(e -> calculateDiscount());
btnRevertRegular.addActionListener(e -> revertRegularMember());
btnRevertPremium.addActionListener(e -> revertPremiumMember());
btnPayDue.addActionListener(e -> payDueAmount());
btnDisplay.addActionListener(e -> displayMembers());
btnClear.addActionListener(e -> clearFields());

```

```

        btnSaveToFile.addActionListener(e -> saveToFile());
        btnReadFromFile.addActionListener(e -> readFromFile());

        // Add buttons to panel
        panel.add(btnAddRegular);
        panel.add(btnAddPremium);
        panel.add(btnActivate);
        panel.add(btnDeactivate);
        panel.add(btnMarkAttendance);
        panel.add(btnUpgradePlan);
        panel.add(btnCalculateDiscount);
        panel.add(btnRevertRegular);
        panel.add(btnRevertPremium);
        panel.add(btnPayDue);
        panel.add(btnDisplay);
        panel.add(btnClear);
        panel.add(btnSaveToFile);
        panel.add(btnReadFromFile);

        panel.setVisible(true);
        return panel;
    }

    // Method to add a regular member
    public void addRegularMember() {
        try {

```

```

String memberId = txtId.getText().trim();
String name = txtName.getText().trim();
String location = txtLocation.getText().trim();
String phone = txtPhone.getText().trim();
String email = txtEmail.getText().trim();

String gender = rbMale.isSelected() ? "Male" : (rbFemale.isSelected() ?
"Female" : "");

if (gender.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Please select a gender!", "Error",
JOptionPane.ERROR_MESSAGE);
    return;
}

LocalDate dob = LocalDate.of(
    Integer.parseInt(cbYear.getSelectedItem().toString()),
    Integer.parseInt(cbMonth.getSelectedItem().toString()),
    Integer.parseInt(cbDay.getSelectedItem().toString())
);

LocalDate startDate = LocalDate.of(
    Integer.parseInt(cbStartDateYear.getSelectedItem().toString()),
    Integer.parseInt(cbStartDateMonth.getSelectedItem().toString()),
    Integer.parseInt(cbStartDateDay.getSelectedItem().toString())
);

String referral = txtReferral.getText().trim();
RegularMember.Plan plan = (RegularMember.Plan) cbPlan.getSelectedItem();
double paidAmount = Double.parseDouble(txtPaidAmount.getText().trim());

```

```

        RegularMember member = new RegularMember(memberId, name, location,
phone, email,
        gender, dob, startDate, referral, plan, paidAmount);

        members.add(member);

        JOptionPane.showMessageDialog(this, "Regular member added successfully!");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Error adding member: " +
e.getMessage());
    }
}

// Method to add a premium member
private void addPremiumMember() {
    try {
        String memberId = txtId.getText().trim();

        // Check if member ID already exists
        if (findMemberById(memberId) != null){
            JOptionPane.showMessageDialog(this, "Member ID already exists!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        String name = txtName.getText().trim();
        String location = txtLocation.getText().trim();
        String phone = txtPhone.getText().trim();

```



```

String email = txtEmail.getText().trim();

String gender = rbMale.isSelected() ? "Male" : (rbFemale.isSelected() ?
"Female" : "");

if (gender.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Please select a gender!", "Error",
JOptionPane.ERROR_MESSAGE);
    return;
}

```

```

LocalDate dob = LocalDate.of(
    Integer.parseInt(cbYear.getSelectedItemAt(0).toString()),
    Integer.parseInt(cbMonth.getSelectedItemAt(0).toString()),
    Integer.parseInt(cbDay.getSelectedItemAt(0).toString())
);

```

```

LocalDate startDate = LocalDate.of(
    Integer.parseInt(cbStartDateYear.getSelectedItemAt(0).toString()),
    Integer.parseInt(cbStartDateMonth.getSelectedItemAt(0).toString()),
    Integer.parseInt(cbStartDateDay.getSelectedItemAt(0).toString())
);

```

```
String referral = txtReferral.getText().trim();
```

```
String trainer = txtTrainer.getText().trim();
```

```

PremiumMember member = new PremiumMember(memberId, name, location,
phone, email, gender,

```

```
    dob, startDate, referral, trainer);
```

```
members.add(member);
```

```

JOptionPane.showMessageDialog(this, "Premium member added
successfully!");

```

```

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Invalid date format! Use yyyy-MM-dd",
"Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

// Method to activate membership

```

private void activateMembership() {
    String memberId = txtId.getText();
    GymMember member = findMemberById(memberId);

    if (member != null) {
        member.activateMembership();
        JOptionPane.showMessageDialog(this, "Membership activated successfully!");
    } else {
        JOptionPane.showMessageDialog(this, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

```

// Method to deactivate membership

```

private void deactivateMembership() {
    String memberId = txtId.getText();
    GymMember member = findMemberById(memberId);

    if (member != null) {
        member.deactivateMembership();
    }
}

```

```

        JOptionPane.showMessageDialog(this, "Membership deactivated successfully!");
    } else {
        JOptionPane.showMessageDialog(this, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

```

// Method to mark attendance

```

private void markAttendance() {
    try {
        String memberId = txtId.getText();
        LocalDate attendanceDate = LocalDate.of(
            Integer.parseInt(cbStartDateYear.getSelectedItem().toString()),
            Integer.parseInt(cbStartDateMonth.getSelectedItem().toString()),
            Integer.parseInt(cbStartDateDay.getSelectedItem().toString())
        );

        GymMember member = findMemberById(memberId);
        if (member != null) {
            if (member.isActiveStatus()) {
                member.markAttendance();
                JOptionPane.showMessageDialog(this, "Attendance marked successfully
for member ID: " + memberId);
            } else {
                JOptionPane.showMessageDialog(this, "Cannot mark attendance for
inactive member!", "Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }
}

```

```

        } else {
            JOptionPane.showMessageDialog(this, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Invalid date format! Use yyyy-MM-dd",
"Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

```

// Method to upgrade plan
private void upgradePlan() {
    String memberId = txtId.getText().trim();
    GymMember member = findMemberById(memberId);

    if (member instanceof RegularMember) {
        RegularMember regular = (RegularMember) member;
        RegularMember.Plan currentPlan = regular.getPlan();
        RegularMember.Plan[] plans = RegularMember.Plan.values();
        int currentIndex = currentPlan.ordinal();
        if (currentIndex < plans.length - 1) {
            regular.setPlan(plans[currentIndex + 1]);
            txtRegularPrice.setText(String.valueOf(plans[currentIndex + 1].getPrice()));
            JOptionPane.showMessageDialog(this, "Plan upgraded to " +
plans[currentIndex + 1].name());
        } else {
            JOptionPane.showMessageDialog(this, "Already at highest plan!");
        }
    }
}

```

```

    }
    } else {
        JOptionPane.showMessageDialog(this, "Only regular members can upgrade
plans!", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

// Method to calculate discount

```

private void calculateDiscount() {
    String memberId = txtId.getText();
    GymMember member = findMemberById(memberId);

    if (member != null) {
        if (member instanceof PremiumMember) {
            PremiumMember premiumMember = (PremiumMember) member;
            premiumMember.calculateDiscount();

            txtDiscountAmount.setText(String.valueOf(premiumMember.getDiscountAmount()));
            JOptionPane.showMessageDialog(this, "Discount calculated successfully!");
        } else {
            JOptionPane.showMessageDialog(this, "This feature is only for Premium
members!", "Error", JOptionPane.ERROR_MESSAGE);
        }
    } else {
        JOptionPane.showMessageDialog(this, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

```

```

    }

    // Method to revert regular member
    private void revertRegularMember() {
        String memberId = txtId.getText();
        GymMember member = findMemberById(memberId);

        if (member != null) {
            if (member instanceof RegularMember) {
                RegularMember regularMember = (RegularMember) member;
                String reason = txtRemovalReason.getText();
                regularMember.revertMember(reason);

                JOptionPane.showMessageDialog(this, "Regular member reverted
successfully!");
            } else {
                JOptionPane.showMessageDialog(this, "This member is not a Regular
member!", "Error", JOptionPane.ERROR_MESSAGE);
            }
        } else {
            JOptionPane.showMessageDialog(this, "Member not found!", "Error",
JOptionPane.ERROR_MESSAGE);
        }
    }

    // Method to revert premium member
    private void revertPremiumMember() {
        String memberId = txtId.getText().trim();

```

```

GymMember member = findMemberById(memberId);

if (member instanceof PremiumMember) {
    members.remove(member);
    JOptionPane.showMessageDialog(this, "Premium member removed from list.");
} else {
    JOptionPane.showMessageDialog(this, "This member is not a premium member!",
    "Error", JOptionPane.ERROR_MESSAGE);
}
}

```

```

// Method to pay due amount
private void payDueAmount() {
    String memberId = txtId.getText().trim();
    GymMember member = findMemberById(memberId);

    if (member instanceof RegularMember) {
        RegularMember regular = (RegularMember) member;
        double due = regular.getPlan().getPrice() - regular.getPaidAmount();
        if (due > 0) {
            regular.setPaidAmount(regular.getPaidAmount() + due);
            txtPaidAmount.setText(String.valueOf(regular.getPaidAmount()));
            JOptionPane.showMessageDialog(this, "Due amount of " + due + " paid
            successfully.");
        } else {
            JOptionPane.showMessageDialog(this, "No due amount left to pay.");
        }
    }
}

```

```

    }
} else {
    JOptionPane.showMessageDialog(this, "Only regular members have due
amounts!", "Error", JOptionPane.ERROR_MESSAGE);
}
}

// Method to display all members
private void displayMembers() {
    if (members.isEmpty()) {
        JOptionPane.showMessageDialog(this, "No members to display!", "Information",
JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    // Create a new frame to display members
    JFrame displayFrame = new JFrame("Member List");
    displayFrame.setSize(800, 500);
    displayFrame.setLocationRelativeTo(this);

    // Create text area for display
    JTextArea textArea = new JTextArea();
    textArea.setEditable(false);
    textArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));

    // Add member information

```



```

    for (GymMember member : members) {
        String memberType = (member instanceof RegularMember) ? "Regular" :
"Premium";

        textArea.append(member.display() + "\nMembership Type: " + memberType + "\n--
-----\n");
    }

    // Add text area inside a scroll pane
    JScrollPane scrollPane = new JScrollPane(textArea);

    displayFrame.add(scrollPane);
    displayFrame.setVisible(true);
}

// Method to clear all input fields
private void clearFields() {
    txtId.setText("");
    txtName.setText("");
    txtLocation.setText("");
    txtPhone.setText("");
    txtEmail.setText("");
    bgGender.clearSelection();
    cbDay.setSelectedIndex(0);
    cbMonth.setSelectedIndex(0);
    cbYear.setSelectedIndex(0);
    cbStartDateDay.setSelectedIndex(0);
    cbStartDateMonth.setSelectedIndex(0);
}

```

```

        cbStartDateYear.setSelectedIndex(0);
        txtReferral.setText("");
        txtPaidAmount.setText("");
        txtRemovalReason.setText("");
        txtTrainer.setText("");
        txtDiscountAmount.setText("0");
        cbPlan.setSelectedIndex(0);
    }

```

// Method to save member data to file

```

private void saveToFile() {
    try {
        if (members.isEmpty()) {
            JOptionPane.showMessageDialog(this, "No members to save!", "Information",
JOptionPane.INFORMATION_MESSAGE);
            return;
        }

```

```

        FileWriter writer = new FileWriter("MemberDetails.txt");

```

// header

```

        writer.write(String.format("%-5s %-15s %-15s %-15s %-25s %-20s %-10s %-
10.2f %-10d %-15d %-10s %-15s %-15.2f %-15.2f\n",
            "ID", "Name", "Location", "Phone", "Email", "Membership Start Date", "Plan",
            "Price", "Attendance", "Loyalty Points", "Active Status", "Full Payment",
            "Discount Amount", "Net Amount Paid"));
        writer.write("=".repeat(200) + "\n");

```

```

// member information

DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");

for (GymMember member : members) {
    if (member instanceof RegularMember) {
        RegularMember rm = (RegularMember) member;

        writer.write(String.format("%-5s %-15s %-15s %-15s %-25s %-20s %-10s %-10.2f %-10d %-15d %-10s %-15s %-15s %-15s\n",
            rm.getMemberId(), rm.getName(), rm.getLocation(), rm.getPhone(),
            rm.getEmail(),
            rm.getMembershipStartDate().format(formatter), rm.getPlan(),
            rm.calculatePrice(), rm.getAttendanceCount(), rm.getLoyaltyPoints(),
            rm.isActiveStatus() ? "Active" : "Inactive", "N/A", "N/A",
            rm.calculatePrice()));
    } else if (member instanceof PremiumMember) {
        PremiumMember pm = (PremiumMember) member;

        writer.write(String.format("%-5s %-15s %-15s %-15s %-25s %-20s %-10s %-10.2f %-10d %-15s %-10s %-15s %-15.2f %-15.2f\n",
            pm.getMemberId(), pm.getName(), pm.getLocation(), pm.getPhone(),
            pm.getEmail(),
            pm.getMembershipStartDate().format(formatter), "Premium",
            pm.calculatePrice(), pm.getAttendanceCount(), "N/A",
            pm.isActiveStatus() ? "Active" : "Inactive",
            pm.isFullPayment() ? "Yes" : "No", pm.getDiscountAmount(),
            pm.getPaidAmount()));
    }
}

```

```

    }

    writer.close();

    JOptionPane.showMessageDialog(this, "Member details saved to
MemberDetails.txt successfully!");
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this, "Error saving to file: " + e.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
    }

}

// Method to read member data from file
private void readFromFile() {
    try {
        File file = new File("MemberDetails.txt");
        if (!file.exists()) {
            JOptionPane.showMessageDialog(this, "File MemberDetails.txt does not
exist!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Create a new frame to display file contents
        JFrame displayFrame = new JFrame("Member Details from File");
        displayFrame.setSize(800, 500);
        displayFrame.setLocationRelativeTo(this);
    }
}

```

```

// Create text area for display
JTextArea textArea = new JTextArea();
textArea.setEditable(false);
textArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));

// Read file contents
BufferedReader reader = new BufferedReader(new FileReader(file));
String line;
while ((line = reader.readLine()) != null) {
    textArea.append(line + "\n");
}
reader.close();

// Add text area to a scroll pane
JScrollPane scrollPane = new JScrollPane(textArea);
displayFrame.add(scrollPane);

displayFrame.setVisible(true);
} catch (IOException e) {
    JOptionPane.showMessageDialog(this, "Error reading from file: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
}
}

// Helper method to find a member by ID
private GymMember findMemberById(String memberId) {
    for (GymMember member : members) {

```

```
        if (member.getMemberId().equalsIgnoreCase(memberId)) {  
            return member;  
        }  
    }  
    return null;  
}  
  
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        new GymGUI(); // Directly launch GymGUI  
    });  
}  
}
```

